Computer Science Department

Dr. Victor Matos

Chapter 14

More About Classes

Static Members

Friend functions

Copy-Constructor

Overloading Operators

Aggregation

JSON Encoding

Circular References

Serialization / Deserialization

An analogy

- In most colleges there is a **Clock Tower**, it keeps the *official time* that is shared by everybody on campus.
- **Static** variables are *shared data* items, much like the school's official time.
- The official time is a communal unique value, and it is independent of the number of students currently present on the campus.

14.1

Instance and Static Members

C++ **static** Data Members

- In C++, static members of a class are shared among all instances of the class. They belong to the class itself rather than any particular object.

Key Characteristics:

1. **Class-Level Storage:**
   - A static data member is shared by all objects of the class.
   - Only one copy of the static data member exists in memory, regardless of how many objects of the class are created.
2. **Declaration:**
   - Declared inside the class using the static keyword.
3. **Definition:**
   - Must be defined and initialized outside the class.
   - Starting from C++17, static integral **constants** can be initialized inline within the class.
4. **Access:**
   - Can be accessed directly using the class name or through an object of the class.

Example1. C++ **static** Data Members

```cpp
#include <iostream>
using namespace std;

class MyClass {
public:
    static int count;   // Declaration of static data member

    MyClass() {
        count++;        // Increment static count on object creation
    }

    ~MyClass() {
        count--;        // Decrement static count on object destruction
    }
};

// Initialization of the static data member
int MyClass::count = 0;



int main()
{
    cout << "Initial count: " << MyClass::count << endl;

    MyClass obj1, obj2;
    cout << "Count after creating two objects: " << MyClass::count << endl;

    {
        MyClass obj3;
        cout << "Count after creating another object: " << MyClass::count << endl;
    } // obj3 goes out of scope

    cout << "Count after obj3 is destroyed: " << MyClass::count << endl;
    return 0;
}
```

The class variable **count** is fetched without an object's intervention!

```
Initial count: 0
Count after creating two objects: 2
Count after creating another object: 3
Count after obj3 is destroyed: 2
```

C++ **static** Member Functions

- In C++, static members functions are used to fetch static data members.

  Key Characteristics:

  1. **No Object Context:**
     - Static member functions are not tied to a specific object of the class.
     - They can only access static data members or other static functions of the class (not non-static members).
  2. **Declaration:**
     - Declared with the static keyword inside the class.
  3. **Calling:**
     - Can be called using the class name or an object of the class.
  4. **Use Cases:**
     - Useful for utility functions that don't depend on instance-specific data.

Example 2. C++ **static** Member Functions

```cpp
#include <iostream>
#include <sstream>
using namespace std;

class BankAccount {
private:
    int    accountNumber;
    double balance;

    // Static data member: shared by all accounts
    static double interestRate;

public:
    BankAccount(int accNo, double initialBalance)
               : accountNumber(accNo), balance(initialBalance) {}

    // Member function to deposit money
    void deposit(double amount) {
        balance += amount;
    }

    // Member function to withdraw money
    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
        }
        else {
            cout << "Insufficient balance!" << endl;
        }
    }
```

The **BankAccount** class holds a static data member (`interestRate`) and two static functions (`get/set interestRate`).

Example 2. C++ **static** Member Functions

```
    // Static member function to set interest rate
    static void setInterestRate(double rate) {
        interestRate = rate;
    }

    // Static member function to get interest rate
    static double getInterestRate() {
        return interestRate;
    }

    // Summarize account details
    string toString() const {
        stringstream sout;
        sout << "Account Number: " << accountNumber
            << ", Balance: " << balance
            << ", Interest Rate: " << interestRate << "%" << endl;
        return sout.str();
     }
};


// Initialization of the static data member
double BankAccount::interestRate = 3.5; // Default interest rate is 3.5%
```

The **BankAccount** class holds a static data member (`interestRate`) and two static functions (`get/set interestRate`).

Example 2. C++ **static** Member Functions

```cpp
int main() {
    // Creating accounts
    BankAccount account1(101, 500.0);
    BankAccount account2(102, 1000.0);

    // Displaying initial details
    cout << "Initial Account Details:" << endl;
    cout << account1.toString();
    cout << account2.toString();

    // Changing interest rate using the class name
    BankAccount::setInterestRate(4.0);

    // Displaying updated details
    cout << "\nAfter Updating Interest Rate:" << endl;
    cout << account1.toString();
    cout << account2.toString();

}
```

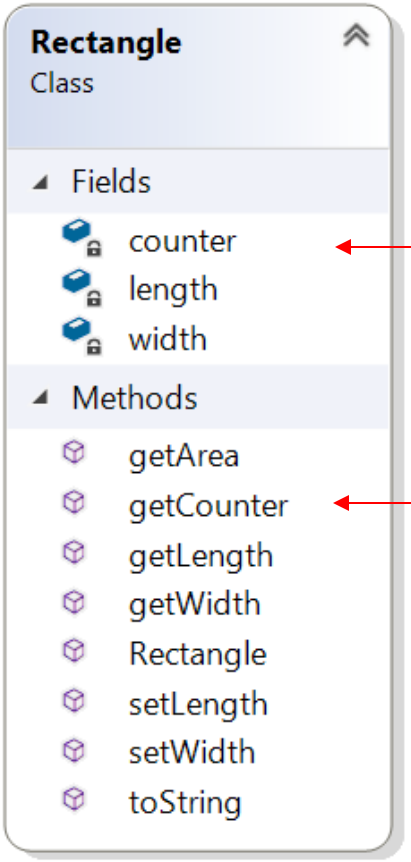
The **BankAccount** class holds a static data member (`interestRate`) and two static functions (`get/set interestRate`).

```
Initial Account Details:
Account Number: 101, Balance: 500, Interest Rate: 3.5%
Account Number: 102, Balance: 1000, Interest Rate: 3.5%

After Updating Interest Rate:
Account Number: 101, Balance: 500, Interest Rate: 4%
Account Number: 102, Balance: 1000, Interest Rate: 4%
```

Friend Functions

- `friend`: is a function or class that is not a member of a class, but has **access to all parts of a class, including private** members.
- In order for a function or class to become a friend of another class, it must be declared as such by the class granting it access.
- A friend function can be a stand-alone function or a member function of another class, or compilation module.
- Use it with caution – *it may compromise security*!
  The real risk is developer discipline — they create a gap in encapsulation that, if not carefully managed, can lead to unintended data exposure or corruption.

Example 3A: Rectangle Class (static & friend)



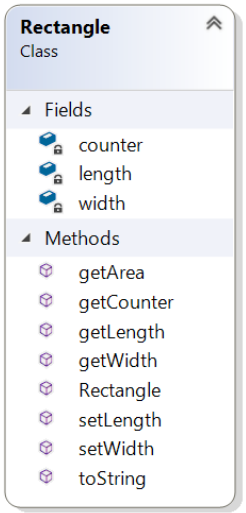
Counter stores how many instances of Rectangle objects have been made so far.

getCounter retrieves the total of rectangles made from the class.

Example 3A: Rectangle Class


```
// DEFINITION SECTION - Includes static member counter,
// get/setCounter, and friend function

class Rectangle {
private:
	double width;
	double length;
	static int counter;

public:
	// Setters
	void	setWidth(double w);
	void	setLength(double len);

	// Getters
	double getWidth();
	double getLength();

	// Constructor & Destructor
	Rectangle(double w, double len);
	~Rectangle();

	//User-defined methods
	string toString() const;
	double getArea();
	static int getCounter();

	// Friend function
	friend void showBox(Rectangle r);
};
```

```
// Preprocessor Declaratives
#pragma once
#include <iostream>
#include <iomanip>
#include <string>
#include <sstream>
using namespace std;
```



Example 3A: Rectangle Class

```
// IMPLEMENTATION SECTION ---------------------------------------------------
int Rectangle::counter = 0;   // The static member variable is traditionally
                              // initialized outside the class
//Getters and Setters
double Rectangle::getWidth()            { return width; }
double Rectangle::getLength()           { return length; }
void   Rectangle::setWidth(double w)    { width = w; }
void   Rectangle::setLength(double len) { length = len; }

//Constructors & Destructor
Rectangle::Rectangle(double w = 0, double len = 0) {
      width = w;  length = len;
      counter++;
}

Rectangle::~Rectangle() { counter--; }
//User-Defined Methods
double Rectangle::getArea() { return width * length; }

string Rectangle::toString() const {
      ostringstream sout;
      sout << " " << this << "  Rectangle [" << " Length:" << length
           << ", Width:" << width << " ]";
      return sout.str();
}

// Static member function
int Rectangle::getCounter() { return counter; }

// Friend function (Observe that it does not use scope resolution
// notation Rectangle::) This code could be defined OUTSIDE the class !
void showBox(Rectangle box) {
      cout << "Box: " << box.length << ", " << box.width << endl;
}
```

showbox is a stand-alone function. Observe that its name is NOT prequalified as **Rectangle::showBox**

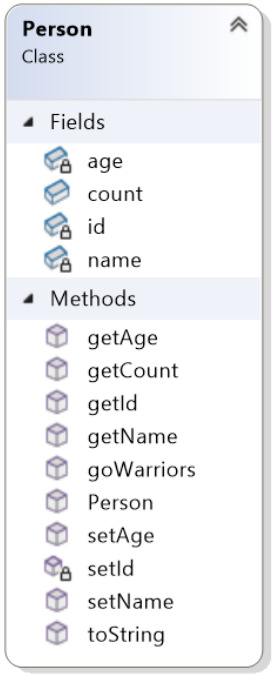
Example 3A: Rectangle Class

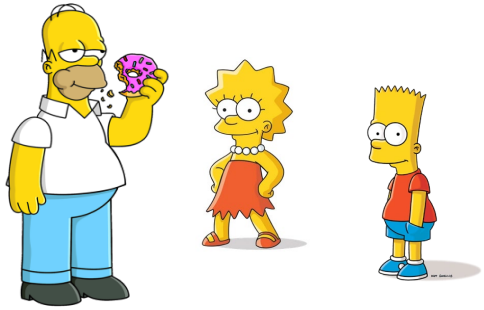
```
// Example of a class holding static data and function members
#include <iostream>
#include <iostream>
#include "Rectangle.h"
using namespace std;

//// Friend function
//// It could be written here instead of the class implementation module!
//void showBox(Rectangle box) {
//cout << "Boxx: " << box.length << ", " << box.width << endl;
//}

int main()
{
	// How many boxes have been made so far?
	cout << "Total boxes: " << Rectangle::getCounter() << endl;

	// Define two Rectangle objects.
	Rectangle box1(10, 10);   // width = 10, length = 10
	cout << box1.toString() << endl;
	cout << "Total boxes: " << Rectangle::getCounter() << endl;

	Rectangle box2(20, 20);  // width = 20, length = 20
	cout << box2.toString() << endl;
	cout << "Total boxes: " << Rectangle::getCounter() << endl;

	// Call the friend function
	showBox(box1);
	showBox(box2);
}
```

```
Total boxes: 0
000000CCCBF2F418  Rectangle [ Length:10, Width:10 ]
Total boxes: 1
000000CCCBF2F448  Rectangle [ Length:20, Width:20 ]
Total boxes: 2
Box: 10, 10
Box: 20, 20
```

Example 3B: Person Class

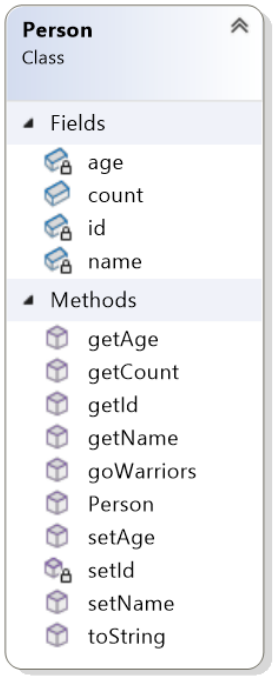
- The following pure-inline `Person` class stores a person's ID, name, and age.
- It includes mutators, accessors, a condensed constructor, and a `toString`() method for object stringification.
- A static data member, `count`, tracks the number of `Person` objects created.
- Each person's ID is assigned using the current `count` value.
- The class also defines a static function, `goWarriors`(), to cheer for ECC sport teams,
- Finally, the class declares a friend function, `bestMan`(), which can access the private members of `Person` objects.





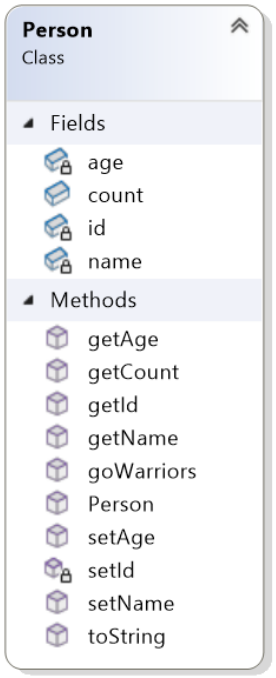
Example 3B: Person Class


```
class Person
{
private:
    string name;
    int    age;
    int    id;

    //Private member function
    void setId() { this->id = 1000 + count; }

public:
    //Static data member and function
    static int count;

    static void goWarriors() {
        //Cheer for El Camino College Warriors
        cout << "Go ECC Warriors!" << endl;
    }

    //Mutator and accessor functions
    void setName(string name) { this->name = name; }
    void setAge(int age)      { this->age = age; }

    string getName()  const { return name; }
    int    getAge()   const { return age; }
    int    getId()    const { return id; }
    int    getCount() const { return count; }

    //Condensed constructor with default values for name and age
    //Note that the id is set by the application (not the user!)
    Person(string name = "No Name", int age = 0)  {
        setName(name);
        setAge(age);
        setId();
        count++;
    }
```



Example 3B: Person Class


```
    //User-defined method - Stringify function
    string toString() const
    {
        stringstream sout;
        sout << this << " Person[ID: " << getId()
            << ", Name: " << getName()
            << ", Age : " << getAge() << "]";
        return sout.str();
    }

    //Declaring a friend function
    friend void bestMan(Person& p);

};

//Initialize the static data member
int Person::count = 0;
```



Example 3B: Person Class


```
#include "Person.h"

//An implementation of the friend function
void bestMan(Person& p) {
    cout << "Ladies and gentlemen, I am Barney Gumble, \n"
        << "I'll tell you a secret about my best friend "
        << p.name << ",\nI bet you didn't know that he is "
        << p.age  << " years old and loves donuts " << endl;
}

int main()
{
    //Calling static data member and function
    cout << "How many people are there? " << Person::count << endl;

    //Function is invoked using the class name
    Person::goWarriors();

    //Create some people
    Person p1("Homer", 39);
    Person p2("Lisa", 8);
    Person p3("Bart", 10);

    //Show the people
    for (Person p : {p1, p2, p3}) {
        cout << p.toString() << endl;
    }

    //Let's make more noise
    p1.goWarriors();
    p2.goWarriors();
    p3.goWarriors();

    //Using the friend function
    bestMan(p1);
}
```

```
How many people are there? 0


Go ECC Warriors!

000000C6204FF3B8 Person[ID: 1000, Name: Homer, Age : 39]
000000C6204FF3B8 Person[ID: 1001, Name: Lisa, Age : 8]
000000C6204FF3B8 Person[ID: 1002, Name: Bart, Age : 10]

Go ECC Warriors!
Go ECC Warriors!
Go ECC Warriors!

Ladies and gentlemen, I am Barney Gumble,
I'll tell you a secret about my best friend Homer,
he is 39 years old and loves donuts!
```

14.4

Background

Member-wise Assignment

Copying a Class - Memberwise Assignment

Definition

In C++, **memberwise copy** refers to the default mechanism used to initialize one class object with data from another object. This happens when:

- The **copy constructor** is invoked, and no user-defined version is available.
- The **assignment operator** (=) is used, and no custom assignment operator is provided.

Protocol

The compiler performs memberwise assignment by **copying each non-static data member of the source object to the corresponding data member of the target object.**

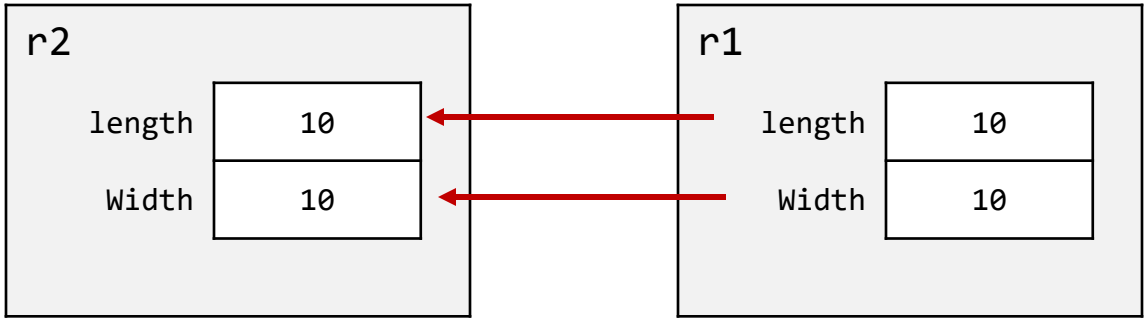
When is it used?

Example 1. Object initialization.	Example 2. Assignment	Example 3. Value Parameter passing & function object returning
`Rectangle r2 = r1;` `Rectangle r3(r1);`	`r2 = r1;`	`function(r1);`

Memberwise Assignment

How Memberwise-Copy Works

- For non-static **primitive types** (like int, double, string, etc.), the compiler performs a simple bitwise copy of their values.
- For **pointers**, only the pointer's value (i.e., the memory address) is copied, not the object it points to.
- **Static members** are **not copied** because they are shared across all objects of the class.
- If a **data member is itself a class**, the compiler calls that class's copy constructor (during construction) or copy assignment operator (during assignment).



Example 4. Memberwise Assignment

Define two rectangles, copy the second rectangle into the first, modify the first.

```
// Memberwise Copy (Shallow Copy)
#include "Rectangle.h"
using namespace std;

int main()
{
	// Define two Rectangle objects.
	Rectangle box1(10, 10);
	cout << "box1 " << box1.toString() << endl;

	Rectangle box2(20, 20);
	cout << "box2 " << box2.toString() << endl;

	// Assign the members of box1 to box2 (default shallow copy).
	box2 = box1;

	// Display each object's width and length again.
	cout << "box1 " << box1.toString() << endl;
	cout << "box2 " << box2.toString() << endl;

	//change the dimensions of box1
	box1.setLength(15);
	box1.setWidth(25);

	//show both rectangles
	cout << "box1 " << box1.toString() << endl;
	cout << "box2 " << box2.toString() << endl;

}
```

```
box1  000000A75054F9C8  Rectangle [ Length:10, Width:10 ]
box2  000000A75054F9F8  Rectangle [ Length:20, Width:20 ]

box1  000000A75054F9C8  Rectangle [ Length:10, Width:10 ]
box2  000000A75054F9F8  Rectangle [ Length:10, Width:10 ]

box1  000000A75054F9C8  Rectangle [ Length:15, Width:25 ]
box2  000000A75054F9F8  Rectangle [ Length:10, Width:10 ]
```

14.4

Copy Constructors

The C++ Rule of Three

If a class manages resources (like dynamically allocated memory via pointers), you'll likely need to define three special member functions:

1. **Destructor:**

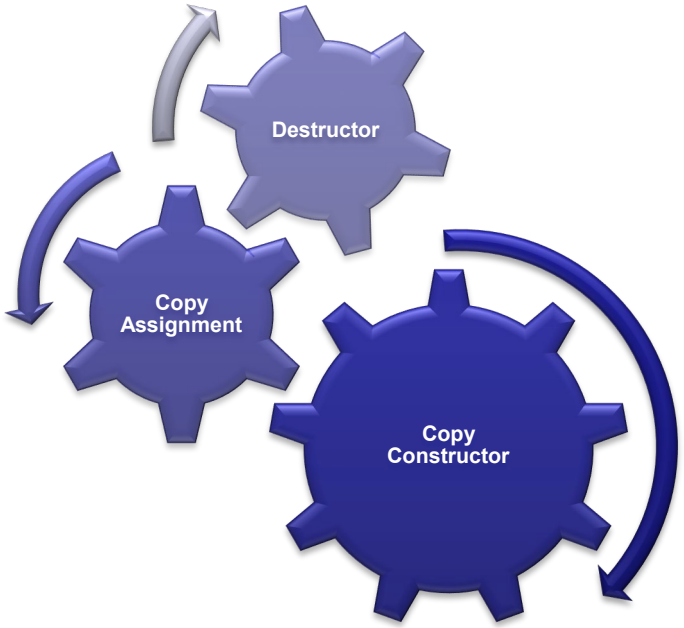
   Responsible for releasing any resources the object owns (e.g., deleting dynamically allocated memory).

2. **Copy Constructor:**

   This function is called when a new object is created as a *deep copy* of an existing object.

3. **Copy Assignment Operator:**

   It handles properly deep-copying the data, and releasing any old data that the left-hand side object may have pointed to.



Copy Constructors

What is it?

A **copy constructor** is a special constructor that is called whenever a new object is initialized from an existing object.

```
SomeClass(const SomeClass& otherObject) {...}
```

- The default copy constructor produces a **clone** of a given object by copying field-by-field the source's data (**memberwise assignment**).
- The default copy constructor works fine in **most** cases.
- However, there are situations in which a **truly independent** clone is wanted. In those cases, memberwise operations **should not** be used.

Copy Constructors

When is the Copy Constructor Used?

```
Rectangle r2(r1);
Rectangle r2 = r1;
function( Rectangle rectangleParam )
```

- An object is passed (by-value) to a function.
- An object is returned from a function.
- A new object is initialized from an existing object.

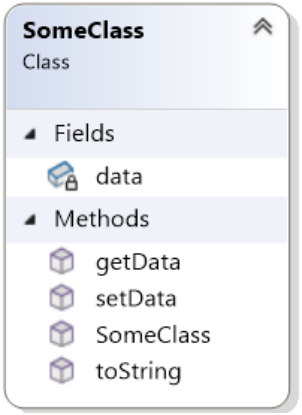
Copy Constructors

When a default constructor is not sufficient?

- The default copy constructor performs a shallow, memberwise initialization, copying each data member from the source object one by one.
- For objects containing **raw pointers** or **heap-managed resources**, this does not produce a **truly independent clone** — both objects will share the same underlying memory.

Example 5A. Default Copy Constructor

```
//Observe that the class has a pointer (int* data) and no custom copy constructor
//(Therefore, the default copy constructor is used - which MAY NOT be what you want!)
class SomeClass
{
private:
    int* data;

public:
    SomeClass(int newData = 0)
    {
        data = new int;     //Create a pointer to dynamic memory
        *data = newData;    //Data is moved to allocated memory
    }

    ~SomeClass() {
        cout << "Destroying " << this
            << " holding: " << this->getData() << " stored at: " << data << endl;
        //delete data;        //Will throw an error if shared with another object
    }

    int  getData() const { return *data; }

    void setData(int newData) { *data = newData; }

    string toString() const {
        ostringstream sout;
        sout << " SomeClass[ data: " << data << ", *data: " << *data
            << ", this: " << this << " ]";
        return sout.str();
    }
};
```



Problem:

- If the originating class has a pointer and it lacks a custom copy-constructor, the clones will not be independent.
- A shallow copy of the source object will be applied.
- The source object and its clone will share the same data value up to the end of the app.

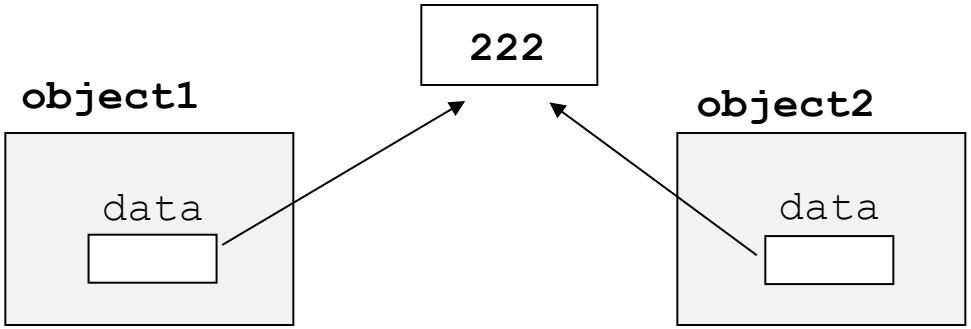
Poor solution *– Needs a custom copy constructor and operator=*

Example 5A. Default Copy Constructor

Using the **default copy constructor** (memberwise copy) with objects referencing dynamic memory.

```
SomeClass object1(111);                                    // Original object
cout << " object1 " << object1.toString() << endl;          // 111

SomeClass object2(object1);                                // Clone
cout << " object2 " << object2.toString() << endl;          // 111

object2.setData(222);      //This assignment on object2 will unintentionally change object1
cout << " object1 " << object1.toString() << endl;          // 222  (original)
cout << " object2 " << object2.toString() << endl;          // 222  (clone)
```

When object2 is created its constructor is not called. Instead, field-by-field assignment takes place, copying object1's member variables into object2.




```
object1  SomeClass[ data: 00000275B0EE7B50, *data: 111, this: 00000079CF0FF718 ]
object2  SomeClass[ data: 00000275B0EE7B50, *data: 111, this: 00000079CF0FF738 ]
object1  SomeClass[ data: 00000275B0EE7B50, *data: 222, this: 00000079CF0FF718 ]
object2  SomeClass[ data: 00000275B0EE7B50, *data: 222, this: 00000079CF0FF738 ]
```

Example 5B. Custom Copy Constructor
A partial remedy to clone dependency

- Add to **SomeClass** your custom version of the copy-constructor (and class destructor.)
- It must refer to the object on the right side of the assignment as a *reference* instance.
- Use the new operator to dynamically create fresh memory, assign it to the object under construction.
- Providing a copy-constructor overrides the membership default copy behavior.

```
//programmer-defined copy constructor (DEEP COPY)
SomeClass(const SomeClass &otherObj)
{
     //request a new memory location
     data = new int;
     //copy on the new cell the data from the other object
     *data = *(otherObj.data);
}

//Destructor - releases memory allocated to the copy-constructor
~SomeClass() {
    delete data;
    data = nullptr;
}
```

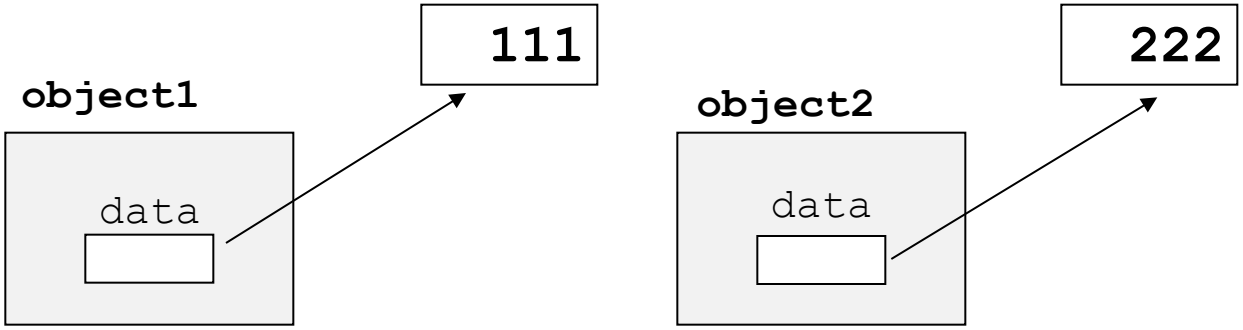
Usage

```
SomeClass obj2(obj1);    //OK
SomeClass obj2 = obj1;   //OK

SomeClass obj3;
obj3 = obj1;             //NOT OK
```

Example 5B. Custom Copy Constructor
A partial remedy to clone dependency

When using a custom-made copy constructor, the source object and its clone will point to different memory locations.

```
SomeClass object1(111);                              // Original object
cout << " object1 " << object1.toString() << endl;  // 111

SomeClass object2(object1);                          // Clone (independent)
cout << " object2 " << object2.toString() << endl;  // 111

object2.setData(222); // This assignment will not affect the original object

cout << " object1 " << object1.toString() << endl;  // 111  (unchanged)
cout << " object2 " << object2.toString() << endl;  // 222  (changed)
```




```
object1  SomeClass[ data: 00000275B0EE7B50, *data: 111, this: 00000079CF0FF718 ]
object2  SomeClass[ data: 00000275B0EE7B50, *data: 111, this: 00000079CF0FF738 ]
object1  SomeClass[ data: 00000275B0EE7B50, *data: 111, this: 00000079CF0FF718 ]
object2  SomeClass[ data: 00000275B0EE7B50, *data: 222, this: 00000079CF0FF738 ]
```

Advise 1

How to make a MUCH better Custom Copy Constructor

1. Overload the **operator=** (see next section.)
2. Use the redefined **operator=** in the making of your copy constructor (see **Point2D** class example - later).

Advantages

You will successfully execute statements such as

```
Student s2(s1);          //Custom Copy-constructor's responsibility

Student s2 = s1;         //Custom Copy-constructor's responsibility

s2 = s1;                 //Operator= responsibility

function(s2);            //Passing an object by value
```

Advise 2

Write Guaranteed Read-Only Accessors

- All your accessors should include the `const` modifier after the parameter list.

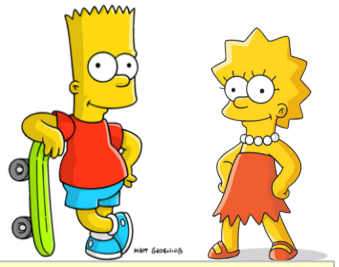
Example.

Instead of `string getName() { ... }`

It should be `string getName() const { ... }`

Why?
The `const` idiom ensures that while the method examines the object to extract some data, it promises not to alter anything.

Example 6: Using a Copy-Constructor



The **Person** class holds person's name and a pointer to a URL showing information about the person.

Person
Class

- Fields
  - name
  - pictureUrl
- Methods
  - ~Person
  - getName
  - getPictureUrl
  - Person (+ 1 overload)
  - setName
  - setPictureUrl
  - toString

```
//Create Bart's object
Person p1("Bart Simpson",
          new string("https://simpsonswiki.com/wiki/Bart_Simpson"));

//Clone Bart's object (Lisa, now you are an exact copy of your brother!)
Person p2 = p1; // Copy constructor invoked
cout << "p1: " << p1.toString() << endl;    // Bart's details
cout << "p2: " << p2.toString() << endl;    // Bart's details

// No way! - Modify the clone to acquire Lisa's details
p2.setName("Lisa Simpson");
p2.setPictureUrl("https://simpsonswiki.com/wiki/Lisa_Simpson");

cout << "p1: " << p1.toString() << endl;    // Bart's details
cout << "p2: " << p2.toString() << endl;    // Lisa's details
```

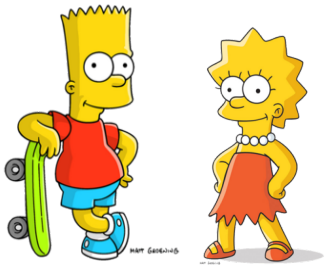
HEAP

```
P1
      name: Bart Simpson
pictureUrl: ABEF0000
```

```
P2
      name:  Lisa Simpson
pictureUrl: ABEF0024
```

ABEF0000	**https://simpsonswiki.com/wiki/Bart_Simpson**
ABEF0024	**https://simpsonswiki.com/wiki/Lisa_Simpson**

Example 6: Using a Copy-Constructor



```
class Person
{
private:
    string  name;
    string* pictureUrl;

public:
    //Accessor and Mutator methods
    string getName() const              { return name; }
    string getPictureUrl() const        { return *pictureUrl; }
    void   setName(string nameValue)    { name = nameValue; }
    void   setPictureUrl(string picUrl) { *pictureUrl = picUrl; }

    //Normal Condensed Constructor
    Person(string nameValue = "n.a.", string* picUrl = nullptr) {
        name = nameValue;
        pictureUrl = new string(*picUrl);
    }

    //Copy-Constructor
    Person(const Person& otherPerson) {
        name = otherPerson.name;
        pictureUrl = new string(*otherPerson.pictureUrl);
    }

    //Destructor
    ~Person() { delete pictureUrl; pictureUrl = nullptr;}

    //toString method
    string toString() const {
        ostringstream sout;
        sout << " Person[ name: " << name << ", pictureUrl: " << *pictureUrl
            << ", this: " << this << " ]";
        return sout.str();
    }
};
```

14.5

Operator Overloading

Why Overload Operators?

Definition:

Overloading allows C++ operators (+, -, *, <<, etc.) to have user-defined meanings for classes.

- **The Goal:** To create a **more intuitive** and readable class interface.

Comparison of Addition Styles:

- **Functional (Purist):** `d3 = d1.add(d2);`
- **Overloaded (Natural):** `d3 = d1 + d2;` → *Much easier to read in complex expressions!*

Key Insight:

Overloading doesn't change what the computer does; it changes how the **programmer** perceives the operation. Use it when the symbol has a clear, logical meaning (like adding two distances).

The Syntax of Operator Overloading

- **The Function Name:**
  Use the keyword `operator` immediately followed by the symbol you wish to redefine.
  - `operator+` → Overloads the **Addition** operator
  - `operator=` → Overloads the **Assignment** operator
  - `operator<<` → Overloads the **Stream Insertion** operator
- **Placement:**
  The declaration and implementation typically belong within the class being overloaded.
- **The Compiler's View:**
  When the compiler sees `a + b`, it internally interprets it as a function call:

```
a.operator+(b);
```

Overloading Operators

Rules and Restrictions

- **Fixed Arity:** You cannot change the number of operands an operator takes.
  Example: A binary + must remain binary; you cannot overload it to take three arguments.
- **No New Symbols:** You can only overload existing C++ operators. You cannot create brand-new operators (like ** or <>).
- **Precedence & Associativity:** These are fixed by the language and cannot be altered by overloading.

The "Un-overloadables": The following operators **cannot** be redefined:

Operator	Name
`.`	Member Access (Dot operator)
`*`	Pointer to Member Access
`::`	Scope Resolution
`?:`	Ternary Conditional
`Sizeof`	Object Size
`Typeid`	Object Type

Operator Overloading

Overloading the **=** operator.

```
SomeClass& operator= (const SomeClass& rightSideValue)
```

- The operator is called through the object on the left side of the assignment.

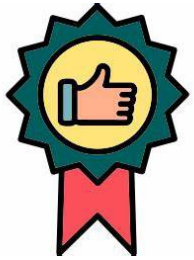
Operator Overloading

Overloading the **=** operator.

```
SomeClass& operator=(const SomeClass& rightSideValue)
```

- Using **reference** & makes our code *more efficient.*
- Using **const** is considered a good practice because it prevents the operator function from modifying the parameter **rightSideValue**.
- It requires that our getter-methods include the `'const'` modifier.

Overloading Operators - Best Practice



- Consider the **Person** class holds the *name* and *age* attributes.
- We may implement the class using any of the following two versions of the **operator<**

```
// Style 1 – Non-Member friend function (Best Practice)
friend bool operator< (const Person& p1, const Person& p2) {
   return p1.name < p2.name;
}
```

```
//Style 2 - Member Function (not always the best choice!)
bool operator< (const Person& other) const {
   return this->name < other.name;
}
```

Benefits of the Non-Member Friend Version

- Symmetric and consistent with how comparison operators are expected to behave.
- Enables implicit conversions on both sides of the operator.
- Works well with STL containers (e.g., `std::set`, `std::map`) that require `operator<`.

Friendly Operator Overloading

When to Overload an Operator Using Friend-Style?

- **Friend function Style** is helpful for overloading operators when *at least one of the operands is not an object of the class itself.*
- This is especially important when the non-member operand appears on the left-hand side of the operator (i.e., **lhs operator rhs**).

Examples

```
Rational r(1, 2);       // Represents the fraction 1/2

cout << r;              // cout is not a Rational => overload <<
                        // using the 'friend' style

r = 5 + r;              // 5 is an integer, not a Rational =>
                        // overload + using 'friend' style
```

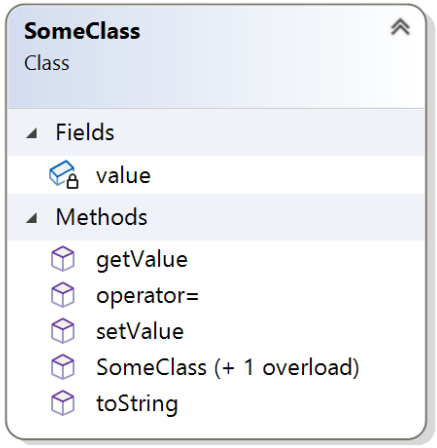
Friendly Operator Overloading

Solution. Add the following methods to the Rational class.

```
//Overload << to move a Rational to an Output-Stream
friend ostream& operator<<(ostream& sout, const Rational& r) {
    sout << r.toString();
    return sout;
}

// Overloaded + operator (non-member friend function, works on int + Rational example)

friend Rational operator+(int lhs, const Rational& rhs) {

    return Rational(lhs * rhs.den + rhs.num, rhs.den);

}


// Overload the + operator (member function, works on Rational + Rational)

Rational operator+(const Rational& other) const {

    int newNum = this->num * other.den + other.num * this->den;

    int newDen = this->den * other.den;

    return Rational(newNum, newDen); // simplify will be called in the constructor

}
```

Example 7. Operator Overloading

Example. Overload the SomeClass **operator=**
Add the following method to the class in Example 5B.
Observe we **must** use the Member-Function style here!

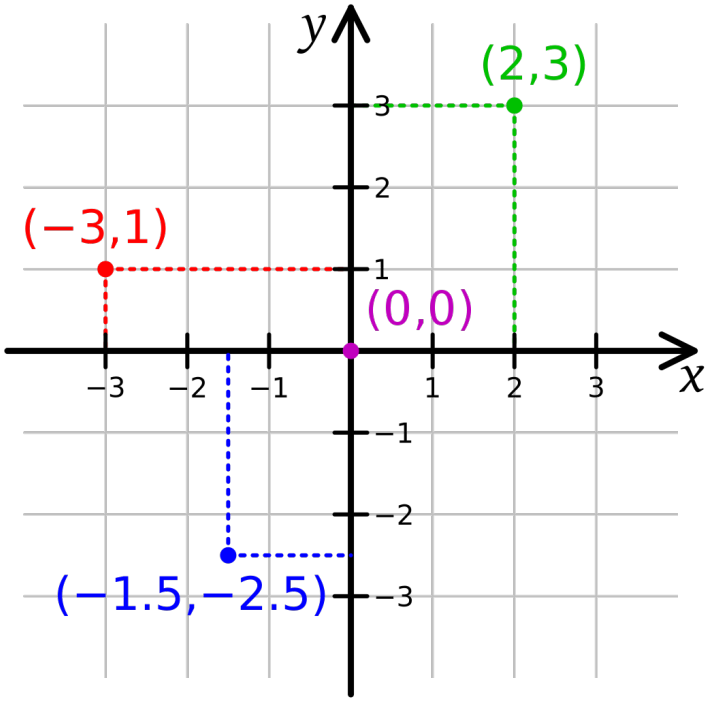


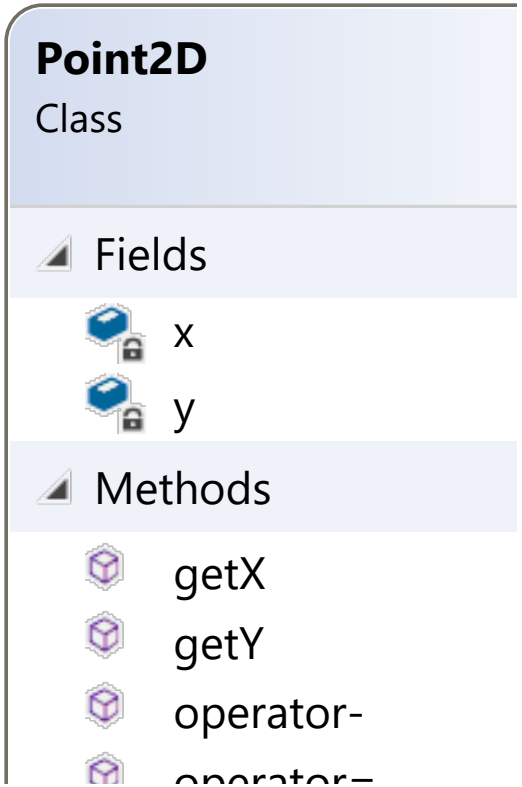

```
class SomeClass
{
private:
    int* value;

public:

    SomeClass& operator=(const SomeClass& rhs)
    {
        if (this != &rhs)                   // Avoid self-assignment
        {
            delete value;                   // Free existing memory
            value = new int;                // Allocate new memory
            *value = *(rhs.value);          // Copy the rhs value
        }
        return *this;
    }

};
```

Note. The C++ standard explicitly requires that **operator=**, along with **operator[]**, **operator()**, and **operator->**, must be **non-static member functions**. This is enforced by the compiler.

Example 8: Two-Dimensional Coordinates

The class Point2D represents points on the plane in the form of (x,y) coordinates.
We want to support a deep-copy assignment syntax, such as: point1 = point2;





Example 8: Two-Dimensional Coordinates

- Combining solutions. The following copy-constructor will use our redefined **operator=**

```
#pragma once
#include <sstream>
#include <cmath>
using namespace std;

class Point2D {
private:
    //data member variables
    int x; int y;

public:

    //accessors-mutators
    void setX(int xValue) { x = xValue; }
    void setY(int yValue) { y = yValue; }

    int  getX() { return x; }
    int  getY() { return y; }

    //constructor(s)
    Point2D(int xValue = 0, int yValue = 0) {
        x = xValue; y = yValue;
    }

    //Copy-Constructor (calls overloaded operator= )
    Point2D(const Point2D& other) {
        *(this) = other;
    }
```

Point2D
Class

Fields
- x
- y

Methods
- getX
- getY
- operator-
- operator=

Example 8: Two-Dimensional Coordinates

- Below is an implementations of `operator=` and `operator-` (subtract two points).

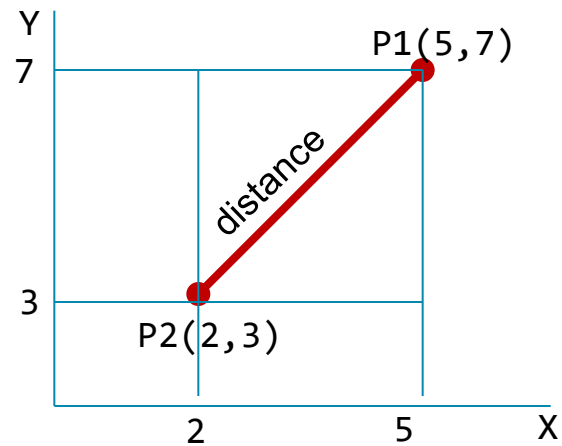
```
    // Redefining operator=
    void operator= (const Point2D& other) {
        this->x = other.x;
        this->y = other.y;
    }

    // Overloaded minus - calculate the distance between two points
    double operator-(const Point2D& other)
    {
        double distance = sqrt(pow((this->x - other.x), 2)
                               + pow((this->y - other.y), 2));
        return distance;
    }

    string toString() const {
        ostringstream sout;
        sout << this << " Point2D [x=" << x << ", y=" << y << "]";
        return sout.str();
    }

};
```

Example 8: Two-Dimensional Coordinates

- Observe how overloaded **operator-** returns a value (distance)

```
int main()
{
    // testing the copy constructor, operator-, and operator=

    Point2D p1(5, 7);       // Define point (5, 7)
    Point2D p2(2, 3);       // Define point (2, 3)

    double dist = p1 - p2;  // distance between p1 and p2

    cout << "Distance " << dist << endl;

    Point2D p3 = p1;        // p3 is an independent clone of p1

    cout << "p1 " << p1.toString() << endl;
    cout << "p3 " << p3.toString() << endl;
}
```

```
Distance 5
p1 000000F05C98F758 Point2D [x=5, y=7]
p3 000000F05C98F7B8 Point2D [x=5, y=7]
```



$$distance = \sqrt{(p2.x - p1.x)^2 + (p2.y - p1.y)^2}$$

Example 9. Returning a Value from an Overloaded Operator

- Overload **operator+** and **operator+=** for **Point2d** objects. Add this new code to Example 8.

```
class Point2D {
//data member variables
private:
      int x; int y;
public:
      ...
      // Redefine operator + for Point2d objects
      Point2D operator+ ( Point2D &other)
      {
            Point2D temp;
            temp.x = this->x + other.x;
            temp.y = this->y + other.y;
            return temp;
      }

      // Redefine += for Point2d objects
      Point2D operator+= ( Point2D & other)
      {
            x += other.getX();
            y += other.getY();
            return *this;
      }
};
```

Point2D
Class

Fields
- x
- y

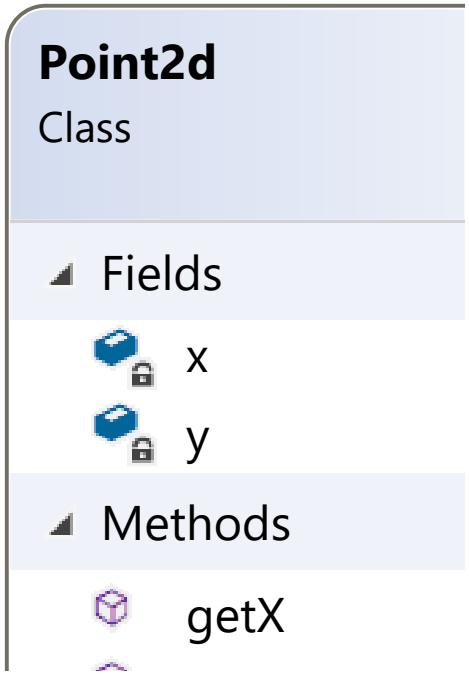
Methods
- getX
- getY
- operator-
- operator+

```
Point2D p4 = p1 + p2;                    // p4 = 5/7 + 2/3 = 29/21
cout << "p4 " << p4.toString() << endl;
p4 += p1;                                // p4 = 29/21 + 5/7 = 44/21
cout << "p4 " << p4.toString() << endl;
```

Example 9. Returning a Value from an Overloaded Operator

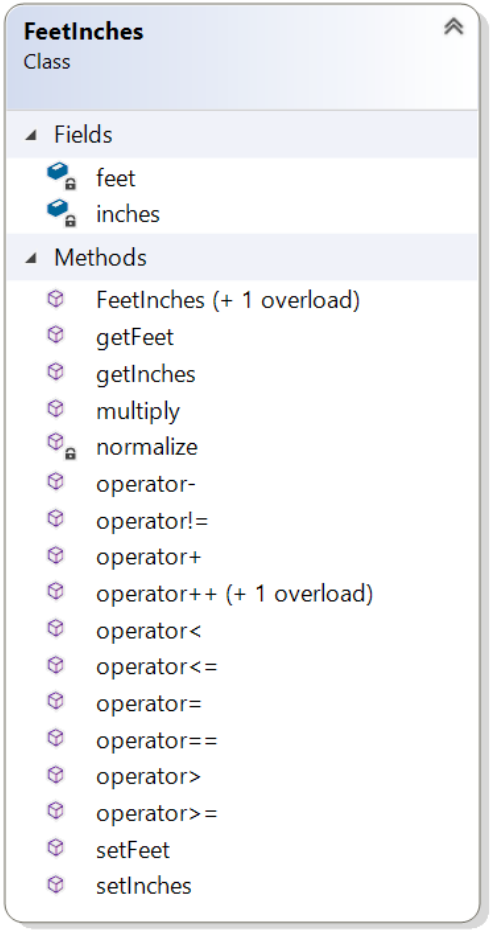
- Overload **operator++** for Point2D objects (prefix and postfix). Add new code to the Example8 code.

```
class Point2d {
//data member variables
private:
      int x; int y;

public:
      ...

      // overload unary ++  (prefix version)
      Point2D operator++ ()
      {
            ++x;
            ++y;
            return *this;
      }

      // overload unary ++  (postfix version – uses dummy arg)
      Point2D operator++ (int dummy)
      {
           Point2D temp = *this;
            x++;
            y++;
            return temp;
      }

};
```




```
++p1;                                   // p1 = p1 + 1 = 5/7 + 1/1  = 12/7
cout << "p1 " << p1.toString() << endl;
p1++;                                   // p4 = p4 + 1 = 12/7 + 1/1 = 19/7
cout << "p1 " << p1.toString() << endl;
```

Case Study: English Distance



Create a class (**FeetInches**) to represent distances using feet and inches.

Overload its common operators.

FeetInches Class - Overloading Operators


```
// (FeetInches.h)
// Example based on
// https://stackoverflow.com/questions/37078649/feetinches-program-overloading-modify-operators
// The current code has been corrected (original program had some flaws)
//-----------------------------------------------------------------------------------------
// This is the specification file for the FeetInches class
#include <iostream>
#pragma once

using namespace std;

class FeetInches
{
private:
    int  feet;         // To hold a number of feet
    int  inches;       // To hold a number of inches
    void normalize();  // Defined in FeetInches.cpp
public:
    // Constructor
    FeetInches(int f = 0, int i = 0) {
        feet = f;
        inches = i;
        normalize();
    }

    // Mutator functions
    void setFeet(int f) { feet = f; }

    void setInches(int i) { inches = i;  normalize(); }
```

FeetInches Class - Overloading Operators


```
    // Accessor functions
    int getFeet() const { return feet; }

    int getInches() const { return inches; }

    // Overloaded operator functions
    FeetInches operator + (const FeetInches&); // Overloaded +
    FeetInches operator = (const FeetInches&); // Overloaded =

    bool operator <= (const FeetInches&);
    bool operator >= (const FeetInches&);
    bool operator != (const FeetInches&);

    FeetInches operator - (const FeetInches&);  // Overloaded -
    FeetInches operator ++ ();                  // Prefix ++
    FeetInches operator ++ (int);               // Postfix ++

    bool operator > (const FeetInches&);       // Overloaded >
    bool operator < (const FeetInches&);       // Overloaded <
    bool operator == (const FeetInches&);      // Overloaded ==

    // copy constructor
    FeetInches(const FeetInches& right)
    {
        feet = right.feet;
        inches = right.inches;
    }
```

FeetInches Class - Overloading Operators


```
    //multiply function
    FeetInches multiply(FeetInches obj)
    {
        FeetInches temp;
        temp.feet = feet * obj.feet;
        temp.inches = inches * obj.inches;
        temp.normalize();
        return temp;
    }


    // Friends
    friend ostream& operator << (ostream&, const FeetInches&);
    friend istream& operator >> (istream&, FeetInches&);

};
```

FeetInches Class - Overloading Operators


```
// Implementation file for the FeetInches class  (FeetInches.cpp)
#include "FeetInches.h"

//*************************************************************
// Definition of member function normalize. This function    *
// checks for values in the inches member greater than       *
// twelve or less than zero. If such a value is found,       *
// the numbers in feet and inches are adjusted to conform    *
// to a standard feet & inches expression. For example,      *
// 3 feet 14 inches would be adjusted to 4 feet 2 inches and *
// 5 feet -2 inches would be adjusted to 4 feet 10 inches    *
// 5 feet and -25 inches becomes 3 feet and 1 inch           *
//*************************************************************

void FeetInches::normalize()
{
    if (inches >= 12)
    {
        feet += (inches / 12);
        inches = inches % 12;
    }
    else if (inches < 0)
    {
        feet -= ((abs(inches) / 12) + 1);
        inches = 12 - (abs(inches) % 12);
    }
}
```

FeetInches Class - Overloading Operators


```
// Overloaded binary = operator.
FeetInches FeetInches::operator = (const FeetInches& right)
{
    // self-assignment guard (do they have the same address?)
    if (this == &right)
        return *this;
    // do the normal copy
    feet = right.feet;
    inches = right.inches;
    return *this;
}

// Overloaded binary + operator.
FeetInches FeetInches::operator + (const FeetInches& right)
{
    FeetInches temp;

    temp.inches = inches + right.inches;
    temp.feet = feet + right.feet;
    temp.normalize();
    return temp;
}
```

FeetInches Class - Overloading Operators


```
// Overloaded binary - operator.
FeetInches FeetInches::operator - (const FeetInches& right) {
    FeetInches temp;
    temp.inches = inches - right.inches;
    temp.feet = feet - right.feet;
    temp.normalize();
    return temp;
}

// Overloaded prefix ++ operator. Causes the inches member to
// be incremented. Returns the incremented object.
FeetInches FeetInches::operator ++ () {
    ++inches;
    normalize();
    return *this;
}

// Overloaded postfix ++ operator. Causes the inches member to be
// incremented. Returns the value of the object before the increment.
FeetInches FeetInches::operator ++ (int) {
    FeetInches temp(feet, inches);
    inches++;
    normalize();
    return temp;
}
```

FeetInches Class - Overloading Operators


```
// Utility function - Converts FeetInches distance to inches
int toInches(const FeetInches& d) {
    return 12 * d.getFeet() + d.getInches();
}

// Overloaded > operator. Returns true if the current object (this)
// is set to a value greater than that of right.
bool FeetInches::operator > (const FeetInches& right) {
    return (toInches(*this) > toInches(right));
}

// Overloaded the <= operator.
bool FeetInches::operator <= (const FeetInches& right) {
    return (toInches(*this) <= toInches(right));
}

// Overloaded the >= operator.
bool FeetInches::operator >= (const FeetInches& right) {
    return (toInches(*this) >= toInches(right));
}

// Overloaded the != operator.
bool FeetInches::operator != (const FeetInches& right) {
    return (toInches(*this) != toInches(right));
}
```

FeetInches Class - Overloading Operators


```
// Overloaded < operator
bool FeetInches::operator < (const FeetInches& right) {
    return (toInches(*this) < toInches(right));
}

// Overloaded == operator
bool FeetInches::operator == (const FeetInches& right) {
    return (toInches(*this) == toInches(right));
}

// Overloaded << operator. Allows cout to directly display FeetInches objects.
ostream& operator<<(ostream& strm, const FeetInches& obj) {
    strm << obj.feet << " feet, " << obj.inches << " inches";
    return strm;
}

// Overloaded >> operator. Allows cin to input directly into FeetInches objects.
istream& operator >> (istream& strm, FeetInches& obj) {
    // Prompt the user for the feet.
    cout << "Feet: ";    strm >> obj.feet;
    // Prompt the user for the inches.
    cout << "Inches: ";  strm >> obj.inches;
    // Normalize the values.
    obj.normalize();
    return strm;
}
```

FeetInches Class - Overloading Operators


```
// TestOverloadedOperators.cpp   (Main - Testing overloaded operators)
#include <iostream>
#include <iostream>
#include "FeetInches.h"
using namespace std;

int main()
{
    FeetInches d1;
    cin >> d1;
    cout << "d1 " << d1 << endl;

    FeetInches d2(2, 10);
    cout << "d2 " << d2 << endl;

    FeetInches d3 = d1 + d2;
    cout << "d3 " << d3 << endl;

    if (d3 <= d1)
        cout << "Yes \n";
    else
        cout << "No \n";

    FeetInches d4 = d3 - d1;
    cout << "d4 " << d4 << endl;
}
```

```
Feet: 5
Inches: 15
d1: 6 feet, 3 inches
d2: 2 feet, 10 inches
d3: 9 feet, 1 inches
No
d4: 2 feet, 10 inches
```

Overloading operator<<

Best Practice

Assume the `operator<<` is introduced inside of the `Rectangle` class as a `friend`.

```
friend ostream& operator<<(ostream& strm, const Rectangle& obj);
```

A possible implementation of the **operator<<** in the Rectangle class is

```
// Overloaded << operator.
// Allows cout to directly display Rectangle objects.
ostream& operator<<(ostream& strm, const Rectangle& obj)
{
   strm << &obj << " Rectangle [Length: " << obj.length
        << " Width: " << obj.width << "]" ;
   return strm;
}
```

14.6

Object Conversion

Implicit Type Conversion (Coercion)

Rule:
C++ automatically converts primitive data types when they are assigned to variables of a different type.

Cases: (Assume the following definition: `int iVal; double dVal;`)

1. **Upcasting (Promotion)**
   Converting a "smaller" type to a "larger" type. This is **safe** and maintains precision.
   Example: `dVal = iVal;`
   Internal Action: The integer 5 becomes the double 5.0.

2. **Downcasting (Demotion)**
   Converting a "larger" type to a "smaller" type. This is **risky** because it often leads to **data loss**.
   Example: `iVal = dVal;`
   Internal Action: The fractional part is **truncated** (not rounded). 5.9 becomes 5.

User-Defined Type Conversion

Goal:

Allow an object of a class to be treated as another type (e.g., converting a distance object to a raw integer).
Requirement: You must define a special **Conversion Operator** function.

The Syntax Rule

Conversion operators have a unique syntax:

- They must be **member functions**.
- They have **no return type** (not even void).
- They should typically be marked **const**.

```
// Conversion function: FeetInches to int
// Note: No return type specified before 'operator'
operator int() const {
    return (feet * 12) + inches;
}
```

Usage in Code

Once defined, the conversion can happen **implicitly** (automatically) or **explicitly**.

```
FeetInches d1(2, 5);                        // 2 feet, 5 inches

// Implicit conversion: d1 is automatically treated as an int
int totalInches = d1;

cout << "Total inches: " << totalInches;    // Output: 29
```

User-Defined Type Conversion

Goal:

Allow an object of a class to be treated as another type (e.g., converting a distance object to a raw integer).
Requirement: You must define a special **Conversion Operator** function.

The Syntax Rule

Conversion operators have a unique syntax:

- They must be **member functions**.
- They have **no return type** (not even void).
- They should typically be marked **const**.

```
// Conversion function: FeetInches to int
// Note: No return type specified before 'operator'
operator int() const {
    return (feet * 12) + inches;
}
```

Usage in Code

Once defined, the conversion can happen **implicitly** (automatically) or **explicitly**.

```
FeetInches d1(2, 5); // 2 feet, 5 inches

// Implicit conversion:
// d1 is automatically treated as an int
int totalInches = d1;

cout << "Total inches: "
     << totalInches; // Output: 29
```

14.7

Aggregation

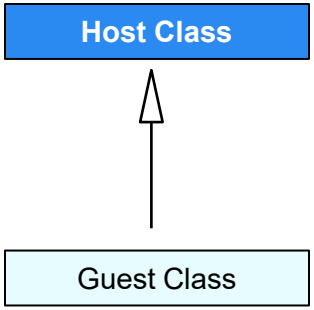
Object Relationships: Composition vs. Aggregation

Core Concept: Both are "Has-A" relationships where a **Host** class uses a **Guest** class as a member variable.

The Key Difference: The difference lies in **ownership** and **lifetime dependency**.

1. Composition ("Part-of")

The Guest object is a fundamental part of the Host.
Strong Ownership: The Host "owns" the Guest.
Lifetime: If the Host is destroyed, the Guest is also destroyed.
Example: A *Room* is part of a *House*. You can't have the room without the house.



2. Aggregation ("Has-a")

The Guest object exists independently of the Host.
It is typically implemented using pointers or references.
Weak Ownership: The Host "references" or "borrows" the Guest.
Lifetime: If the Host is destroyed, the Guest continues to exist.
Example: A *Professor* has a *Computer*. If the professor retires, the computer still exists.

Composition & Aggregation

- A **containment relationship** can be described in simple words as "*an object of one class can own or access the objects of another class*."

- **Examples:**
  - Course *has a* Schedule
  - Course *has a* Professor
  - Course *has a* Textbook
  - Person *has an* Address
  - Car *has an* Engine
  - Car *has an* Infotainment system

Benefits of Aggregation

- ✓ **Reusability**:
  The same object (e.g., Engine, Textbook) can be shared across multiple objects (e.g., multiple Car, Courses instances).

- ✓ **Flexibility**:
  Objects can be replaced or changed dynamically without modifying the container class.

- ✓ **Memory Efficiency**:
  Avoids unnecessary object duplication, especially when managing large objects.

Drawbacks of Aggregation

❌ **Managing External Dependencies:**

The container class must ensure that the aggregated object remains valid during its use.

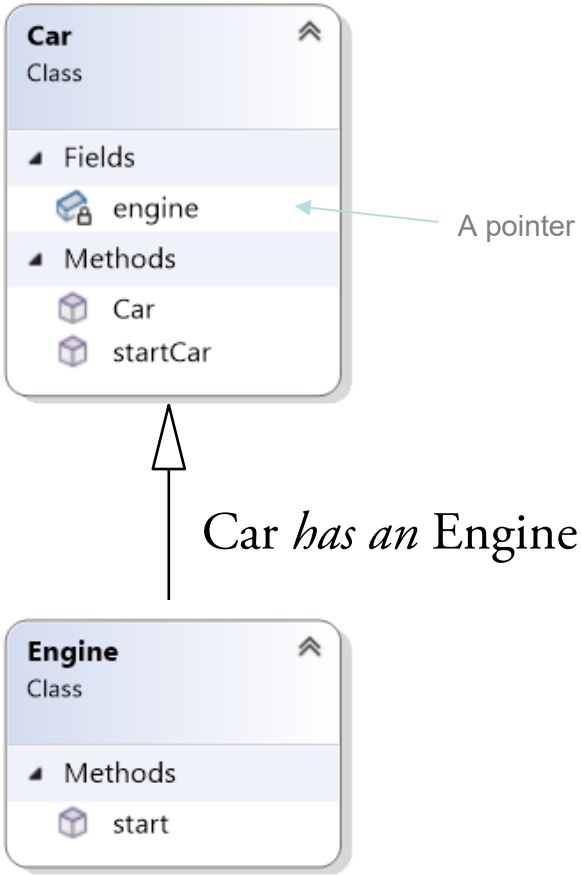
❌ **Potential for Dangling Pointers:**

If the external object (Engine) is deleted while the container (Car) still references it, it can lead to undefined behavior.

❌ **More Complexity:**

Since ownership is not controlled, additional care must be taken to ensure proper lifetime management.

Aggregation – Car-Engine



Aggregation – Car-Engine


```
// C14-Aggregation-Car-Engine.cpp
#include <iostream>
using namespace std;

class Engine {
public:
    void start() {
        cout << "Engine started!" << endl;
    }
};
```

```
// Here Car refers to an external (independent) Engine
class Car {
private:
    Engine* engine;                    // Aggregation: Car does not own Engine

public:
    Car(Engine* eng) : engine(eng) {} // Engine is passed in, not created

    void startCar() {
        if (engine) {
            engine->start();
            cout << "Car is running!" << endl;
        }
        else {
            cout << "No engine found!" << endl;
        }
    }
};
```

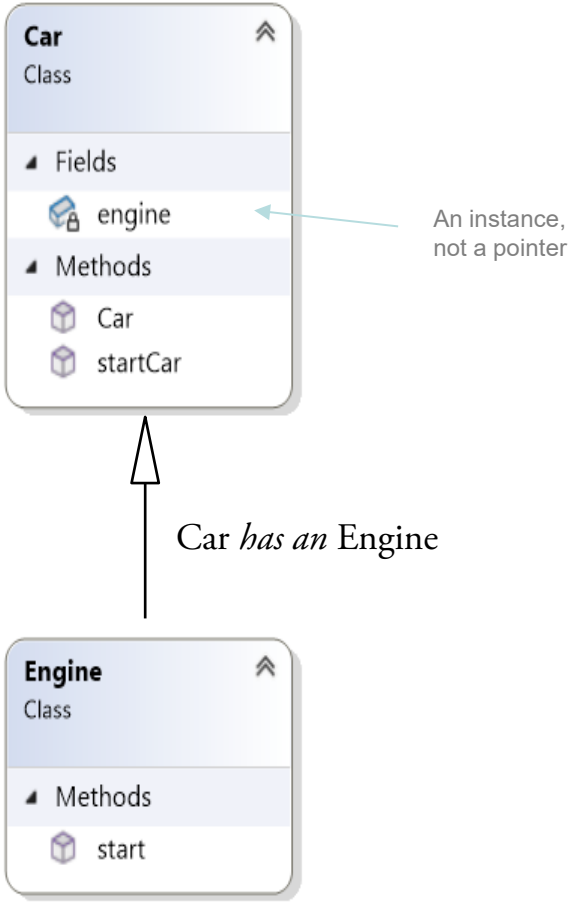
Aggregation – Car-Engine


```
int main() {
    Engine myEngine;        // Engine object created independently
    Car myCar(&myEngine);   // Aggregation: Car receives an existing Engine
    myCar.startCar();

    return 0;
}
```

```
Engine started!
Car is running!
```

Composition

- **Composition** in C++ is a strong form of object containment where one class owns another class's object, meaning the contained object's lifecycle is tied to the container.
- When the container is destroyed, the contained object is also destroyed.
- This is typically implemented using direct object instantiation (not pointers).



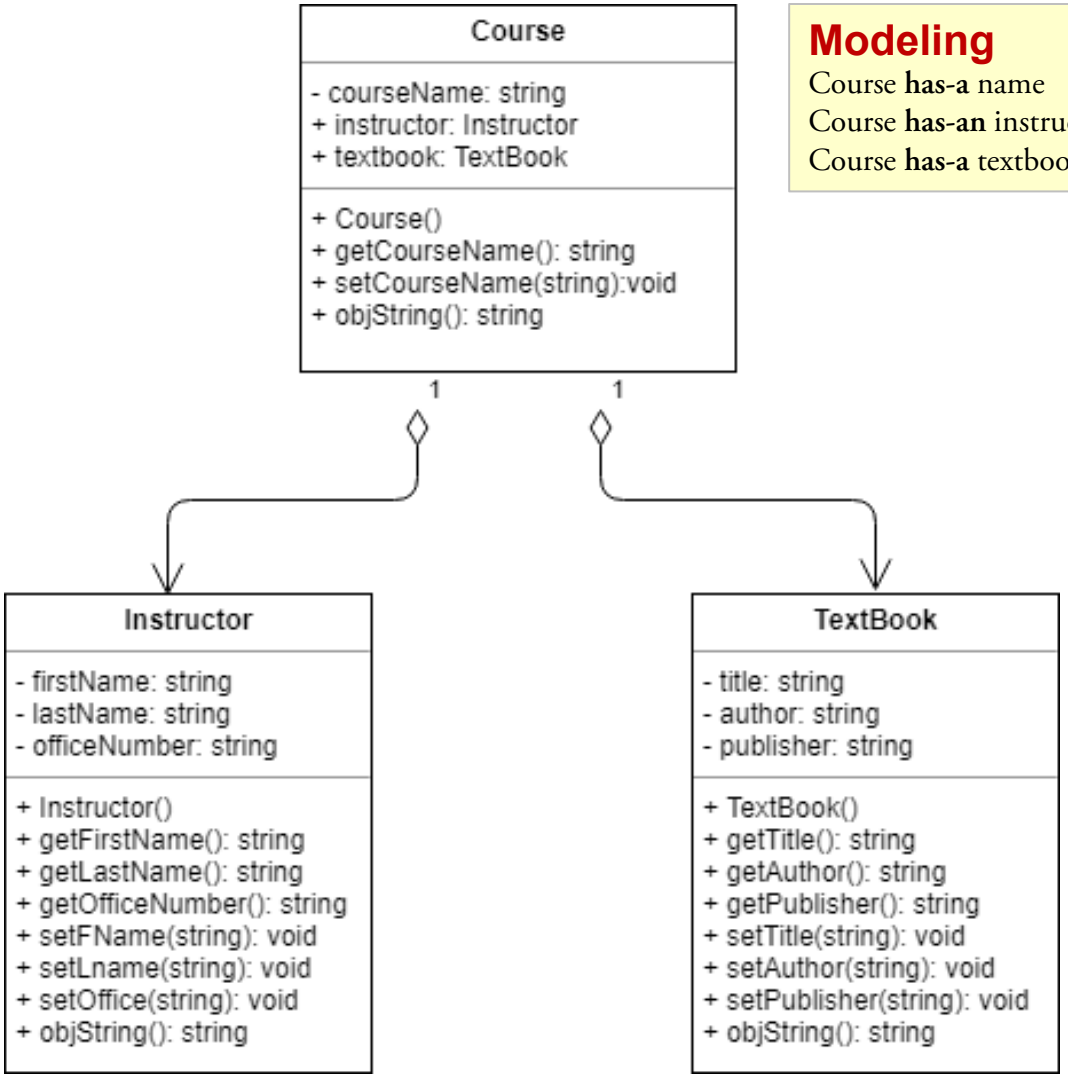
Composition – Car-Engine

```
// C14-Composition-Car-Engine.cpp
#include <iostream>
using namespace std;

class Engine {
public:
    void start() {
        cout << "Engine started!" << endl;
    }
};
```

```
//Here, Car owns Engine, so the engine is created and destroyed with the car.
class Car {
private:
    Engine engine;              // Composition: Car owns Engine

public:
    void startCar() {
        engine.start();
        cout << "Car is running!" << endl;
    }
};


int main() {

    Car myCar;                  // Composition:
                                // Car creates its own Engine
    myCar.startCar();

}
```

Composition – Course-Instructor-Textbook

UML Class Diagram

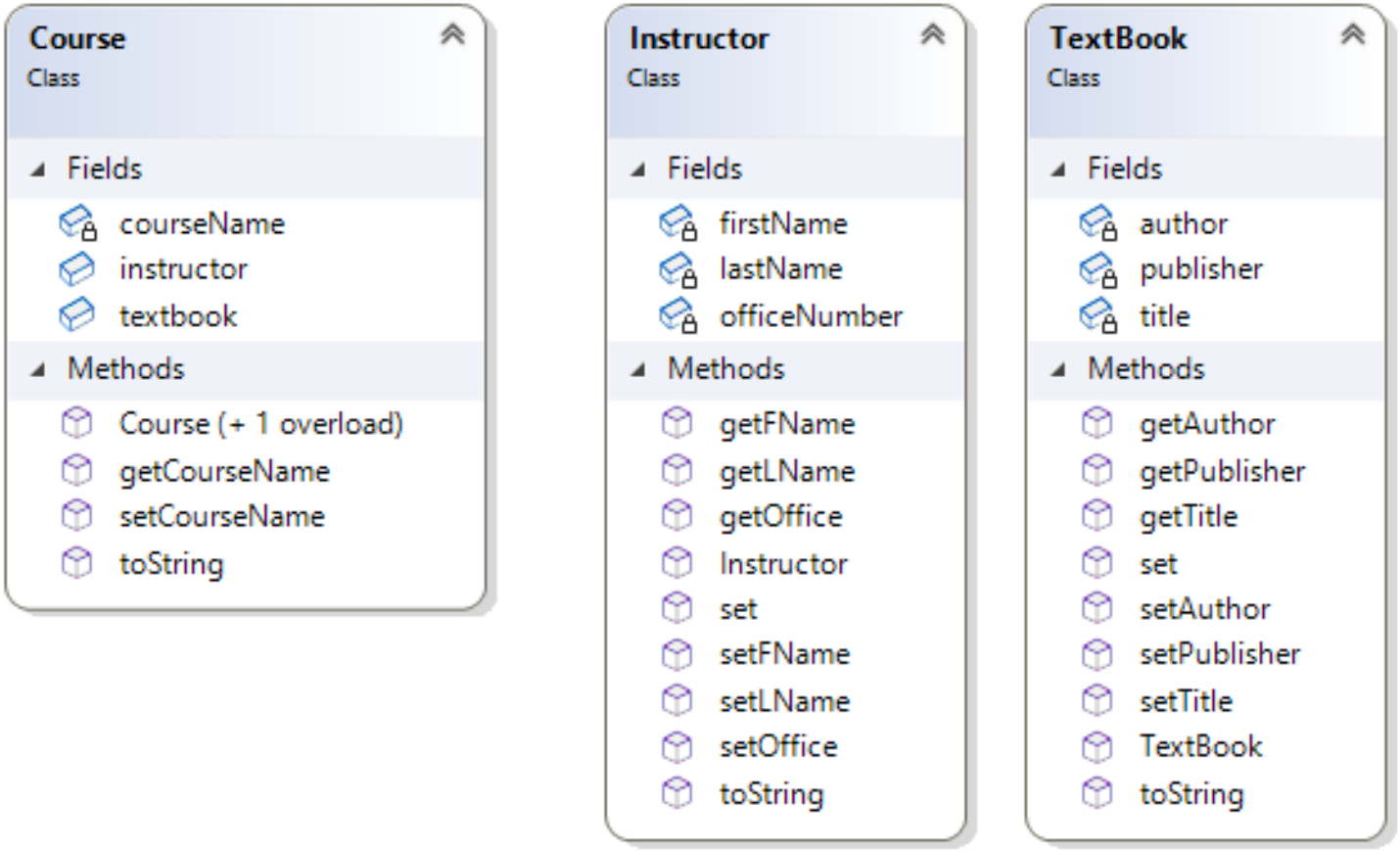


Modeling

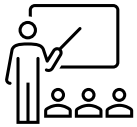
Course **has-a** name
Course **has-an** instructor
Course **has-a** textbook

Composition – Course-Instructor-Textbook

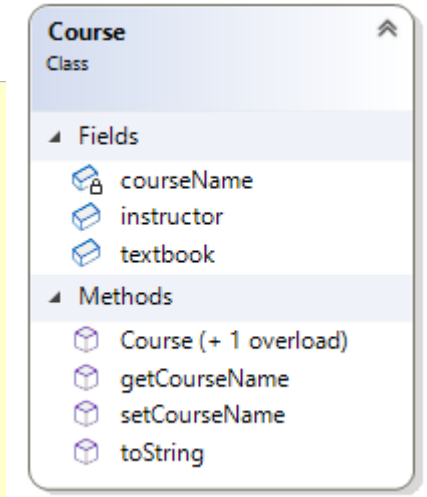
Visual Studio's Class Diagram



Composition – Course



```
#pragma once
#include "MyLibraries.h"
#include "Instructor.h"
#include "TextBook.h"

class Course
{
private:
      string     courseName;          // Course name

public:
      Instructor instructor;          // Instructor
      TextBook   textbook;            // Textbook

      // Constructor
      Course(string course,
             string iLastNameValue, string iFirstNameValue, string iOfficeNumValue,
             string titleValue, string authorValue, string publisherValue);

      Course(string course, Instructor instructorObj, TextBook textBookObj);

      //Getters-Setters
      string getCourseName() { return courseName; }
      void   setCourseName(string value) { courseName = value; }

      // User-Defined Method(s)
      string toString() const;

};
```



Composition – Course

```
//Implementation section ////////////////////////////////////////////////////
Course::Course(string courseNameValue,
               string iLastNameValue, string iFirstNameValue, string iOfficeNumValue,
               string titleValue, string authorValue, string publisherValue)
{      // Assign the course name.
       setCourseName(courseNameValue);

       // Assign the instructor.
       instructor.set(iLastNameValue, iFirstNameValue, iOfficeNumValue);

       // Assign the textbook.
       textbook.set(titleValue, authorValue, publisherValue);
}

Course::Course(string courseValue, Instructor instructorObj, TextBook   textBookObj)
{
       setCourseName(courseValue);
       this->instructor = instructorObj;
       this->textbook = textBookObj;
}

string Course::toString() const
{
       ostringstream sout;
       sout << this;
       sout << " Course [ Name: " << courseName
              << "\n\t\t  " << instructor.toString()
              << "\n\t\t  " << textbook.toString()
              << " ]\n";
       return sout.str();
}
```

Composition – Course *cont.*

```
006FF850 Course [ Name: Quantum Physics
                  006FF86C Instructor [Last name: Einstein, First name: Albert, Office number: MBA342]
                  006FF8C0 Textbook [Title: Theory of Relativity, Author: A. Eisntein, Publisher: McGraw-Hills] ]

006FF784 Course [ Name: Physics I
                  006FF7A0 Instructor [Last name: Newton, First name: Isaac, Office number: MBA344]
                  006FF7F4 Textbook [Title: Intro to Physics, Author: I. Newton, Publisher: Courseware] ]
```

Course
Class

- Fields
  - courseName
  - instructor
  - textbook
- Methods
  - Course
  - getCourseName
  - setCourseName
  - toString

MyLibraries.h

```
#pragma once
#include <string>
#include <sstream>
#include <iostream>
#include <iomanip>
#include <vector>
#include <cmath>
using namespace std;
```

Composition - Instructor

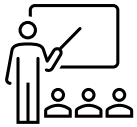
```
#pragma once
#include "MyLibraries.h"

class Instructor
{
private:
    string lastName;        // Last name
    string firstName;       // First name
    string officeNumber;    // Office number

public:
    //Getters-Setters
    string getFName() { return firstName; }
    string getLName() { return lastName; }
    string getOffice() { return officeNumber; }

    void setFName(string value) { firstName = value; }
    void setLName(string value) { lastName = value; }
    void setOffice(string value) { officeNumber = value; }

    // Constructor(s) – observe the use of default values "" (empty string)
    Instructor(string lname = "", string fname = "", string office = "")
    {
        set(lname, fname, office);
    }
```

Instructor
Class

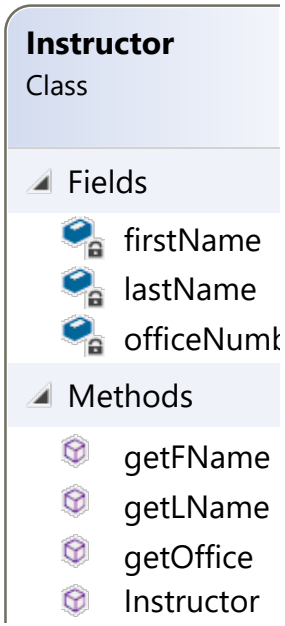
Fields
- firstName
- lastName
- officeNumb

Methods
- getFName
- getLName
- getOffice
- Instructor

Composition - Instructor



```
    // User-defined method(s)
    string toString() const;

    // Utility set function - to abbreviate setting of an instructor
    void set(string lname, string fname, string office);
};

// Implementation section ////////////////////////////////////////////
string Instructor::toString() const
{
    ostringstream sout;
    sout << " Instructor [Last name: " << lastName
        << ", First name: " << firstName
        << ", Office number: " << officeNumber
        << "]";
    return sout.str();
}

// Utility set function - to abbreviate setting of an instructor
void Instructor::set(string lname, string fname, string office)
{
    setLName(lname);
    setFName(fname);
    setOffice(office);
}
```




```
Instructor [Last name: Eistein, First name: Albert, Office number: MBA342]
```

Composition - TextBook

```
#pragma once
#include "MyLibraries.h"

class TextBook
{
private:
    string title;     // Book title
    string author;    // Author name
    string publisher; // Publisher name

public:

    //Getters-Settes
    string getTitle() { return title; }
    string getAuthor() { return author; }
    string getPublisher() { return publisher; }

    void setTitle(string value) { title = value; }
    void setAuthor(string value) { author = value; }
    void setPublisher(string value) { publisher = value; }

    // Constructor(s)
    TextBook(string textTitle = "", string auth = "", string pub = "")
    {
        set(textTitle, auth, pub);
    }
```

TextBook
Class

Fields
- author
- publisher
- title

Methods
- getAuthor
- getPublishe
- getTitle
- set

Composition - TextBook

```
    // User-Defined Methods
    string toString() const;

    // Utility set function - to abbreviate setting of an TextBook
    void set(string textTitle, string auth, string pub);

};
// Implementation section

string TextBook::toString() const
{
    ostringstream sout;
    sout << " Textbook [Title: " << title
        << ", Author: " << author
        << ", Publisher: " << publisher
        << "]";
    return sout.str();
}

void TextBook::set(string textTitle, string auth, string pub)
{
    setTitle(textTitle);
    setAuthor(auth);
    setPublisher(pub);
}
```

TextBook
Class

- Fields
  - author
  - publisher
  - title
- Methods
  - getAuthor
  - getPublishe
  - getTitle
  - set

```
Textbook [Title: Theory of Relativity, Author: A. Eisntein, Publisher: McGraw-Hills]
```

Composition – Testing App

```
// c24-Aggregation.cpp
#include "MyLibraries.h"
#include "Course.h"
#include "Instructor.h"
#include "TextBook.h"

int main()
{
            // Aggregation modeling the following situation:
            // Course HAS-A TextBook, Course HAS-AN Instructor
            // TextBook and Instructor objects survive removal of course

            //TEST1: Create named TextBook and Instructor objects
            Instructor i1("Einstein", "Albert", "MBA342");
            cout << i1.toString() << endl << endl;

            TextBook t1("Theory of Relativity", "A. Eisntein", "McGraw-Hills");
            cout << t1.toString() << endl << endl;

            Course c1("Quantum Physics", i1, t1);
            cout << c1.toString() << endl;

            //TEST2: create a course with anonymous textbook & instructor aggregates
            Course c2("Physics I",
                        "Newton", "Isaac", "MBA344",
                        "Intro to Physics", "I. Newton", "Courseware");

            cout << c2.toString() << endl << endl;
```

Composition – Testing App

```
    //alternatively – we may access individual data items

    cout << c2.getCourseName() << endl;
    cout << c2.instructor.getLName() << endl;
    cout << c2.textbook.getTitle() << endl;

}
```

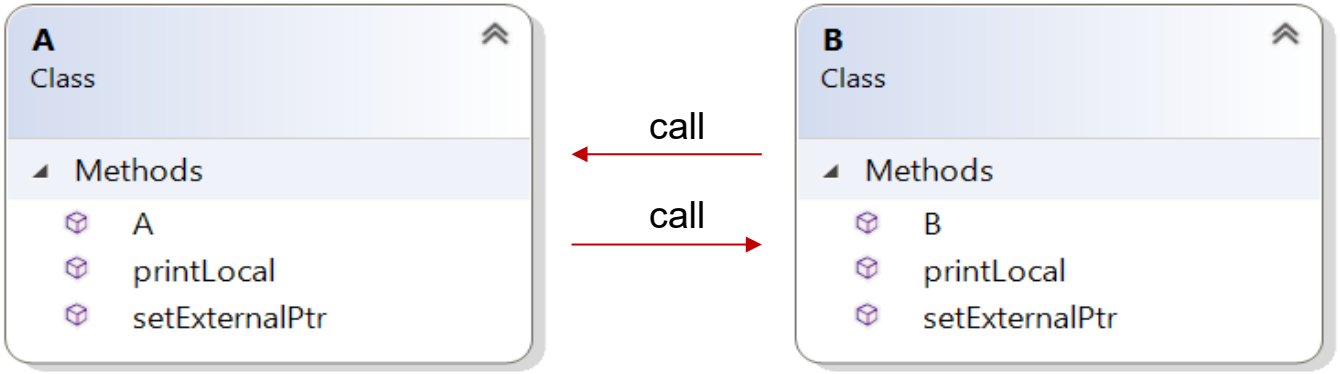
```
Microsoft Visual Studio Debug Console
006FF978 Instructor [Last name: Einstein, First name: Albert, Office number: MBA342]

006FF91C Textbook [Title: Theory of Relativity, Author: A. Eisntein, Publisher: McGraw-Hills]

006FF850 Course [ Name: Quantum Physics
                  006FF86C Instructor [Last name: Einstein, First name: Albert, Office number: MBA342]
                  006FF8C0 Textbook [Title: Theory of Relativity, Author: A. Eisntein, Publisher: McGraw-Hills] ]

006FF784 Course [ Name: Physics I
                  006FF7A0 Instructor [Last name: Newton, First name: Isaac, Office number: MBA344]
                  006FF7F4 Textbook [Title: Intro to Physics, Author: I. Newton, Publisher: Courseware] ]


Physics I
Newton
Intro to Physics
```

Appendix

Circular References

Circular References

- A **circular reference** occurs in C++ when two or more objects hold references (pointers) to each other, preventing proper memory deallocation. This can cause **memory leaks** because neither object gets destroyed when expected.
- In this scenario, you must use the **forward reference strategy** to solve the problem.
- Do not use the pure-inline (Java-Style) declaration mode to define your classes.
- Your classes MUST be implemented using header and implementation sections. All methods referring to objects in the cycle must be placed in the implementation section.



Circular References

Class A [Inline Single Compilation Unit]

```
#pragma once
class B; //forward declaration

//DECLARATION ------------------------------------
class A
{
public:
   A();
   void setExternalPtr(B*);
   void printLocal();
};

//IMPLEMENTATION ---------------------------------
#include "B.h"

A::A() {
   cout << "A constructor called, this: "
       << this << endl;
}

void A::setExternalPtr(B* b) {
   cout << "A gets B* pointer: " << b << endl;
   b->printLocal(); //use pointer to call ext. func.
}

void A::printLocal() {
   cout << "A::printLocal() called by: "
       << this << endl;
}
```

Class B [Inline Single Compilation Unit]

```
#pragma once
class A;  //forward declaration

//DECLARATION ------------------------------------
class B
{
public:
   B();
   void setExternalPtr(A*);
   void printLocal();
};

//IMPLEMENTATION ------------------------------
#include "A.h"

B::B() {
   cout << "B constructor called, this: "
      << this << endl;
}

void B::setExternalPtr(A* a) {
   cout << "B gets A* pointer: " << a  << endl;
   a->printLocal(); //call external function
}

void B::printLocal() {
   cout << "B::printLocal() called by "
       << this << endl;
}
```

Download-Clone Github.com/VMatosECC Circular-Professor-Student

Circular References

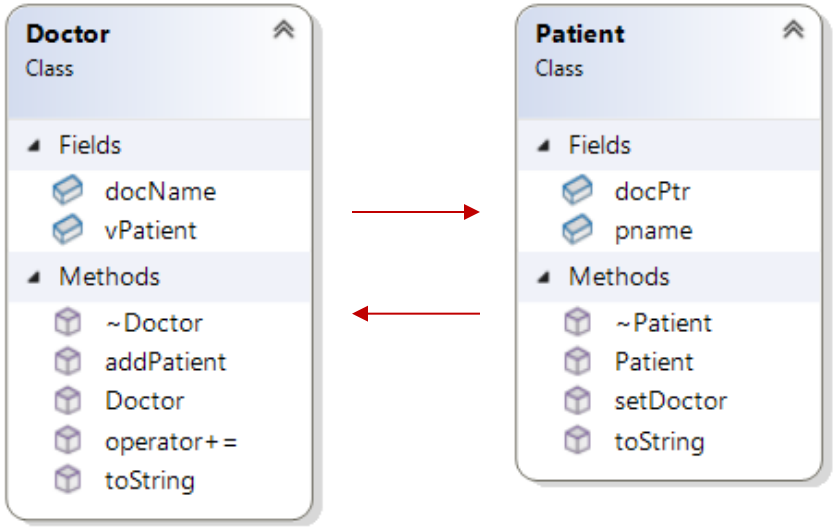
```
// Testing CIRCULAR-REF.cpp
#include "A.h"
#include "B.h"

int main()
{
	A a;
	B b;

	cout << "Local printing\n";
	a.printLocal();
	b.printLocal();

	cout << "\nRemote printing\n";
	a.setExternalPtr(&b);
	b.setExternalPtr(&a);

}
```

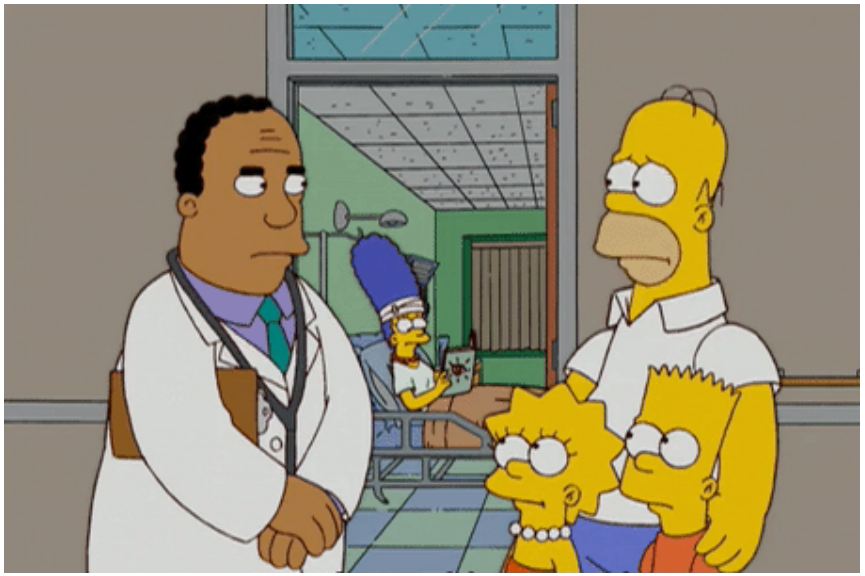
```
A constructor called, this: 0059F85B
B constructor called, this: 0059F84F
Local printing
A::printLocal() called by: 0059F85B
B::printLocal() called by: 0059F84F

Remote printing
A gets B* pointer: 0059F84F
B::printLocal() called by: 0059F84F
B gets A* pointer: 0059F85B
A::printLocal() called by: 0059F85B
```

Circular References

Example of a Circular Relationship

- A doctor has many patients.
- A patient has a single primary care physician.



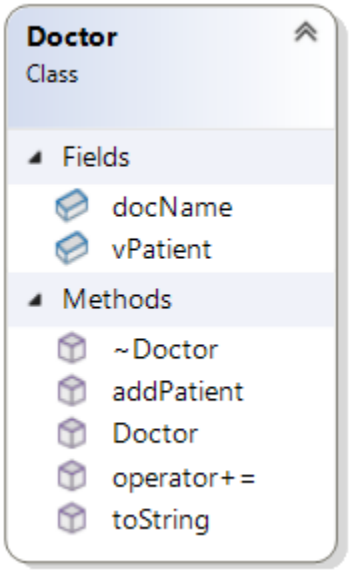


Dr. J. Hibbert is the Simpson's family doctor.
(Cycle: He refers to them, they refer to him)

Download/Clone github.VMatosECC C14-Circular-DoctorPatient

Circular References


```
class Patient;   //forward reference

class Doctor
{
public:
    string docName;
    vector<Patient*> vPatient;
    Doctor(string dname = "Dr. House");
    ~Doctor();
    Doctor& operator+=(Patient* p);
    void addPatient(Patient* p);
    string toString();
};


//--------------------Doctor.cpp IMPLEMENTATION --------------------
#include "Patient.h"

Doctor::Doctor(string dname) : docName(dname) {
    vPatient.clear();
    cout << this << " Doctor created: " << dname << endl;
}
Doctor::~Doctor() { cout << this << " Doctor destroyed: " << docName << endl; }

Doctor& Doctor::operator+=(Patient* p) {
    addPatient(p);
    return *this;
}
void Doctor::addPatient(Patient* p) {
    p->setDoctor(this);
    vPatient.push_back(p);
}

string Doctor::toString() {
    stringstream sout;
    sout << this << " Doctor[ "Name: " << docName;
    sout << ", Patients: " << vPatient.size() << " ]" << endl;
    for (int i = 0; i < vPatient.size(); i++) {
        sout << "\tPatient " << i << ": " << vPatient[i]->pname << endl;
    }
    sout << endl;
    return sout.str();
}
```

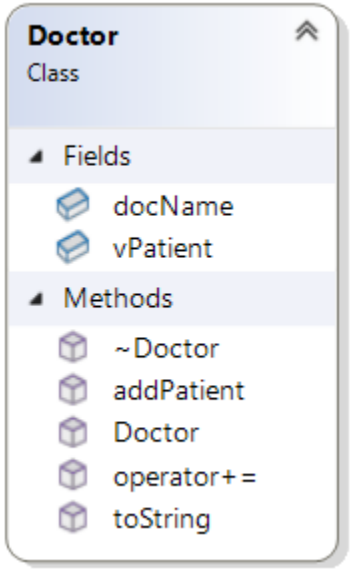



```
#pragma once
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
using namespace std;
```

Why the reference Doctor& ?
It is a common practice in C++ to return a reference from compound assignment operators (like +=, -=, etc.) to allow for chaining and to avoid unnecessary copies.

Circular References


```
#pragma once
using namespace std;
//--------------------Patient.h--------------------
class Doctor;   //forward reference

class Patient
{
public:
    string  pname;
    Doctor* docPtr;

    Patient(string pname = "n.a.");
    ~Patient();

    void setDoctor(Doctor* d);
    string toString();
};


//--------------------Patient.cpp IMPLEMENTATION --------------------
#include "Doctor.h"

Patient::Patient(string pname) {
    this->pname = pname;
    cout << this << " Patient created " << pname << endl;
}

Patient::~Patient() {
    cout << this << " Patient destroyed " << pname << endl;
}

void Patient::setDoctor(Doctor* d) { docPtr = d; }

string Patient::toString() {
    stringstream sout;
    sout << this << " Patient[ Name: " << pname;
    sout << ", Doctor: " << docPtr->docName << " ]" << endl;
    return sout.str();
}
```



Circular References


```cpp
#include <iostream>
#include "Doctor.h"
#include "Patient.h"
using namespace std;



int main() {
    //Create 3 patients and 1 doctor
    Patient* p1 = new Patient("Homer");
    Patient* p2 = new Patient("Lisa");
    Patient* p3 = new Patient("Bart");
    Patient* p4 = new Patient("Marge");

    Doctor* d1 = new Doctor("Dr. Hibbert");

    //Add patients to doctor
    d1->addPatient(p1);
    d1->addPatient(p2);

    *d1 += p3;      //same as d1->addPatient(p3);
    *d1 += p4;      //same as d1->addPatient(p4);

    //Print patients and doctor
    cout << "p1 " << p1->toString() << endl;
    cout << "p2 " << p2->toString() << endl;
    cout << "p3 " << p3->toString() << endl;
    cout << "p4 " << p4->toString() << endl;

    cout << "d1 " << d1->toString() << endl;

    //Clean up
    delete p1;
    delete p2;
    delete p3;
    delete p4;
    delete d1;
}
```

```
000001C1D84A62C0 Patient created Lisa
000001C1D84A6480 Patient created Bart
000001C1D84A6720 Patient created Marge
000001C1D849CDA0 Doctor  created: Dr. Hibbert

p1 000001C1D84A6640 Patient[ Name: Homer, Doctor: Dr. Hibbert ]

p2 000001C1D84A62C0 Patient[ Name: Lisa, Doctor: Dr. Hibbert ]

p3 000001C1D84A6480 Patient[ Name: Bart, Doctor: Dr. Hibbert ]

p4 000001C1D84A6720 Patient[ Name: Marge, Doctor: Dr. Hibbert ]

d1 000001C1D849CDA0 Doctor[ Name: Dr. Hibbert, Patients: 4 ]
        Patient 0: Homer
        Patient 1: Lisa
        Patient 2: Bart
        Patient 3: Marge

000001C1D84A6640 Patient destroyed Homer
000001C1D84A62C0 Patient destroyed Lisa
000001C1D84A6480 Patient destroyed Bart
000001C1D84A6720 Patient destroyed Marge
000001C1D849CDA0 Doctor  destroyed Dr. Hibbert
```

JSON

JavaScript Object Notation

JSON Format

What is JSON?

- JSON, or JavaScript Object Notation, is a lightweight and widely used data interchange format.
- It is easy for humans to read and write and easy for machines to parse and generate.
- JSON is primarily used to transmit data between a server and web application, as an alternative to XML.
- JSON supports text-based serialization-deserialization of OO data.

Key Features of the JSON Standard

- **Data Structure:** Data is stored as a collection of **name:value** pairs, where the name (a string) is followed by a colon and the value.
- **Data Types:** JSON supports various data types, including strings, numbers, objects, arrays, booleans, and null.

Example of JSON Formatting

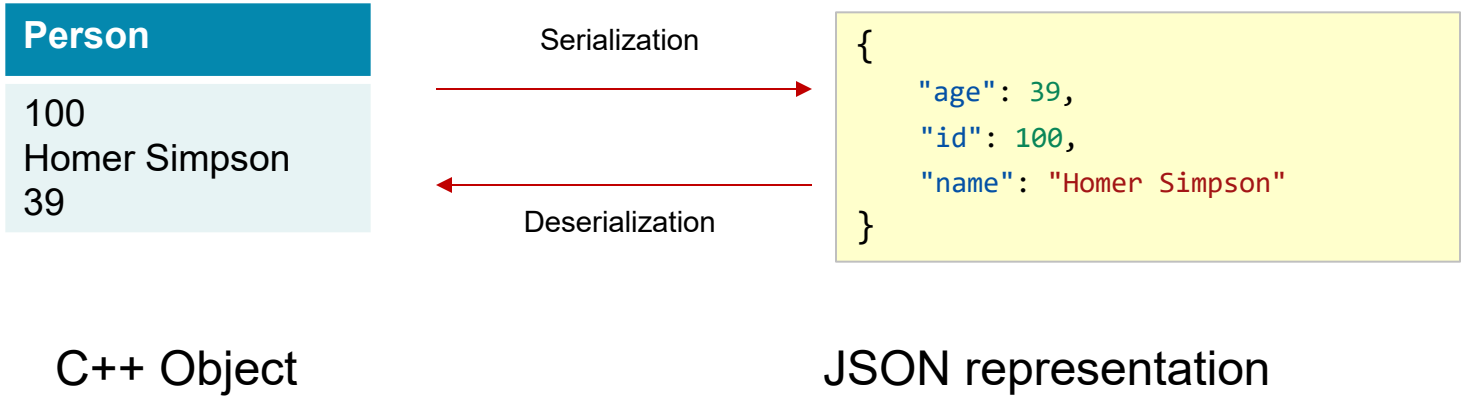
```
{
  "name": "Homer Simpson",
  "age": 39,
  "city": "Springfield",
  "isStudent": false,
  "grades": [59, 70, 68],
  "address": {
     "street": "123 Main St",
     "zipCode": "10001"
   },
   "contact": null
}
```

Syntax Rules

- Data items appear as *key : value* pairs.
- Data items are separated by commas.
- Objects are enclosed in curly braces { }.
- Arrays are enclosed in square brackets [ ].
- Strings are enclosed in double quotes "".
- Numbers, Booleans, and null are written as is.

JSON Serialization - Deserialization

- Serialization is the process of converting an object into a format that can be easily stored or transmitted, such as JSON.
- Deserialization is the reverse process, converting JSON back into an object.



JSON Serialization - Deserialization

Lists

JSON List:

```
[
    {"age":39,"id":100,"name":"Homer Simpson"},
    {"age":36,"id":110,"name":"Marge Simpson"},
    {"age":10,"id":120,"name":"Bart Simpson"}
]
```

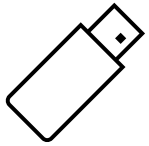
C++ Array

```
Person personList[3] =
{
    Person (100, "Homer Simpson", 39),
    Person (110, "Marge Simpson", 36),
    Person (120, "Bart Simpson", 10),
}
```

JSON Serialization - Deserialization

- **CAUTION** is needed when JSON is used as a **data-interchange** vehicle over the data network, because as any plain-text message, there are some risks of eavesdropping.

- JSON is *language and platform independent,* and it is available for a large number of programming environments [Jackson, Gson, Jayrock, nlohmann RapidJSON].



JSON APIs for C++

Background

C++ doesn't have built-in parsers like some other languages (e.g., Java). However, you can use external libraries (SAX or DOM style) to achieve the same functionality.

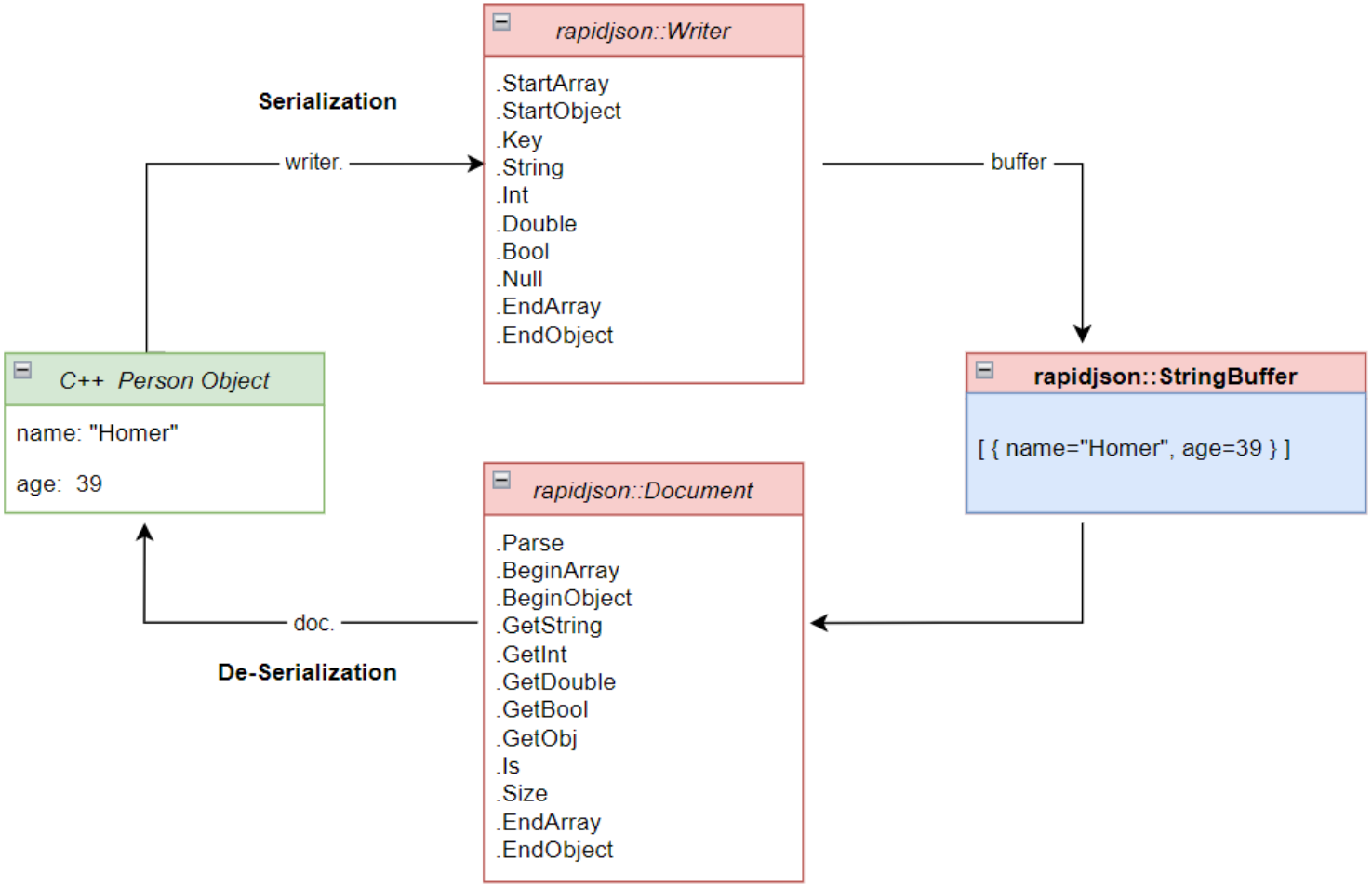
SAX (Simple API for XML) – Example: RapidJson API

- This is an event-driven API where the parser reads through the JSON document and calls specific handler functions when it encounters different elements (e.g., start of an object, key-value pair, end of an array).
- SAX parsing is typically faster and uses less memory than DOM parsing, especially for large documents.

DOM (Document Object Model) – Example: nLohmann/json API

- This API creates an in-memory representation of the entire JSON document as a tree of nodes, which can then be traversed and manipulated.
- DOM parsing provides a more convenient way to access and modify JSON data, allowing for random access to elements and attributes.
- However, it can be slower and use more memory for large documents.

Example1. Using RapidJson



Example1. RapidJson


```cpp
#include <iostream>
#include <vector>
#include "rapidjson/document.h"
#include "rapidjson/writer.h"
#include "rapidjson/stringbuffer.h"
using namespace std;

// ---------------------------------------------------------------------------
class Person {
public:  //Lazy definition
    string name;
    int    age;
    Person(string nameVal="na", int ageVal=0){name = nameVal; age = ageVal;}
};

// ---------------------------------------------------------------------------
// Function to encode a vector of Person objects to JSON format.
// You have to 'manually' encode the object (SAX style), one element at the time.
// ---------------------------------------------------------------------------
string EncodePersonsToJson(const vector<Person>& persons) {
    rapidjson::StringBuffer buffer;
    rapidjson::Writer<rapidjson::StringBuffer> writer(buffer);

    writer.StartArray();
    for (const Person& person : persons) {
        writer.StartObject();
            writer.Key("name");
            writer.String(person.name.c_str());
            writer.Key("age");
            writer.Int(person.age);
        writer.EndObject();
    }
    writer.EndArray();

    return buffer.GetString();
}
```

```
C:\
├── VSWorkspace
│   ├── RapidJsonApp
│   │   ├── Driver.cpp
│   │   └── Driver.exe.txt
│   │   ├── rapidjson
│   │   │   ├── document.h
│   │   │   ├── writer.h
│   │   │   └── stringbuffer.h
```

RapidJson Links
Download: https://sourceforge.net/projects/rapidjson.mirror/
Documentation: https://rapidjson.org/md_doc_encoding.html

Example1. RapidJson


```
// Function to decode JSON into a vector of Person objects
vector<Person> DecodeJsonToPersons(const string& json) {
    vector<Person> persons;

    // Sequentially parse and decode the Json string having the structure
    // Start-Array, Start-Object, Object, End-Object, ... , End-Array
    // and the Objects are made of pairs <key,value> holding name and age.
    rapidjson::Document document;
    document.Parse(json.c_str());

    //verify the string represents a repeating-group
    if (!document.IsArray()) {
        std::cerr << "JSON is not an array" << std::endl;
        return persons;
    }
```

Example1. RapidJson


```
    //Extract one cell at-the-time. Each cell is a Person object
    for (rapidjson::SizeType i = 0; i < document.Size(); ++i) {

        const rapidjson::Value& personObject = document[i];
        //Make sure the object holds the "name" and "age" members
        if (personObject.IsObject()
            && personObject.HasMember("name") && personObject.HasMember("age")) {
            //next two statements retrieve the name and age values
            const rapidjson::Value& nameValue = personObject["name"];
            const rapidjson::Value& ageValue = personObject["age"];
            //if the types agree with those of Person(string, int) create the object
            if (nameValue.IsString() && ageValue.IsInt()) {
                Person person;
                person.name = nameValue.GetString();
                person.age = ageValue.GetInt();
                persons.push_back(person);
            } else {
                std::cerr << "Invalid JSON format for a person object" << std::endl;
            }
        } else {
            std::cerr << "Invalid JSON format for a person object" << std::endl;
        }
    }

    return persons;
}
```

Example1. RapidJson


```
int main() {
    // Create some Person objects
    vector<Person> persons = {
        Person("Homer", 39),
        Person("Bart", 10),
        Person("Lisa", 9),
    };

    // Transform the vector<Person> container to JSON format (SAX style)
    string encodedJson = EncodePersonsToJson(persons);
    cout << "EncodedJson string:\n\t " << encodedJson << endl;

    // Decode the JSON formatted data back into a vector of Person objects
    vector<Person> decodedPersons = DecodeJsonToPersons(encodedJson);

    // Print the decoded Person objects
    cout << "Decoded Json string:\n";
    for (const Person& person : decodedPersons) {
        std::cout << "\t Person [Name: " << personObj.name
                  << ", Age: " << personObj.age << "]\n";
    }
}
```

```
encodedJson:
        [{"name":"Homer","age":39},{"name":"Bart","age":10},{"name":"Lisa","age":9}]

Decoded Json string:
        Person [Name: Homer, Age: 39]
        Person [Name: Bart, Age: 10]
        Person [Name: Lisa, Age: 9]
```

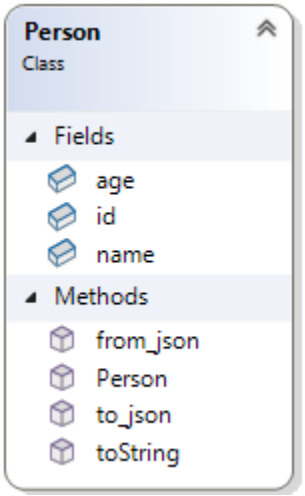
Example2. Using nlohmann Library

to_json: Serialize an Object (Assume all attributes are public)

```
Person p(100, "Homer Simpson", 39);

json j = json{ {"id", id}, {"name", name}, {"age", age} };
```

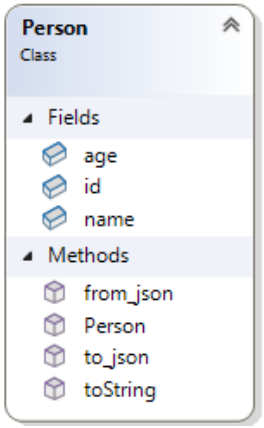
from_json: Deserialize a JSON Object j Representing a Person

```
Person dp(j["id"].get<int>(), j["name"].get<string>(), j["age"].get<int>());
```



Example2. Using nlohmann Library

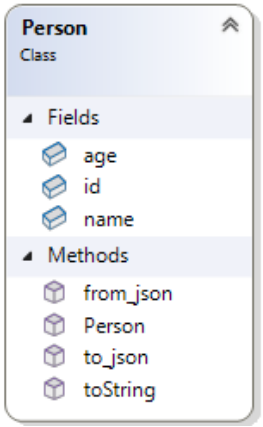

```
class Person {
public:
    int    id;
    string name;
    int    age;

    //Lazy constructor
    Person(int id=0, const string& name="n.a.", int age=0)
          : id(id), name(name), age(age) {}

    // Serialize Person object to JSON
    json to_json() const {
        json j = json{ {"id", id}, {"name", name}, {"age", age} };
        return j;
    }

    // Deserialize JSON to Person object
    static Person from_json(const json& j) {
        return Person(j["id"].get<int>(), j["name"].get<string>(), j["age"].get<int>());
    }

    //Stringify the Person object
    string toString() const {
        stringstream sout;
        sout << "Person[Id: " << id << ", Name: " << name << ", Age: " << age << "]";
        return sout.str();
    }
};
```



Example2. Using nlohmann Library

Create three Person Objects, serialize & deserialize them.

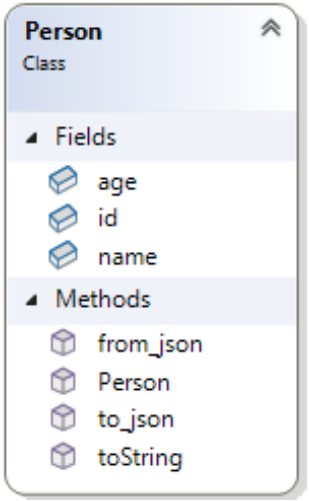
```
// Create Person objects
Person homer(100, "Homer Simpson", 39);
Person marge(110, "Marge Simpson", 36);
Person bart(120, "Bart Simpson", 10);

// Serialize Person objects to JSON
json homer_json = homer.to_json();
json marge_json = marge.to_json();
json bart_json = bart.to_json();

// Print JSON representation (dump(4) indent 4 spaces)
cout << "Homer JSON: " << homer_json.dump(4) << endl;
cout << "Marge JSON: " << marge_json.dump(4) << endl;
cout << "Bart JSON: " << bart_json.dump(4) << endl;

// Deserialize JSON to Person objects
Person new_homer = Person::from_json(homer_json);
Person new_marge = Person::from_json(marge_json);
Person new_bart = Person::from_json(bart_json);
```



Example2. Using nlohmann Library

Create three Person Objects, serialize & deserialize them.

```
Homer JSON: {
    "age": 39,
    "id": 100,
    "name": "Homer Simpson"
}
Marge JSON: {
    "age": 36,
    "id": 110,
    "name": "Marge Simpson"
}
Bart JSON: {
    "age": 10,
    "id": 120,
    "name": "Bart Simpson"
}
```

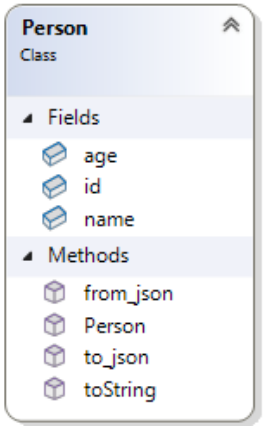


Example2. Using nlohmann Library

Manage the parts of a JSON Person object

```
//Optional - Manage the parts of a JSON object
cout << "Homer's id:     " << homer_json["id"] << endl;
cout << "Homer's name:   " << homer_json["name"] << endl;
cout << "Homer's name:   " << homer_json["name"].get<string>() << endl;
cout << "Homer's age:    " << homer_json["age"] << endl;
cout << "Homer's age:    " << homer_json["age"].get<int>() << endl; //redundant get<int>()
```

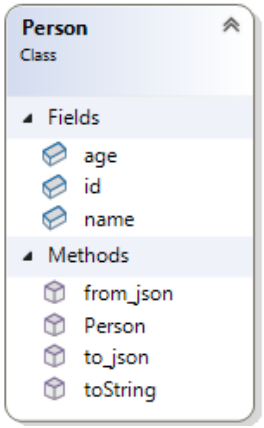
```
Homer's id:     100
Homer's name:   "Homer Simpson"
Homer's name:   Homer Simpson
Homer's age:    39
Homer's age:    39
```



Example2. Using nlohmann Library

Create a list of objects. Write the list to a file. Read the list from the file.

```
    //Make a list of Person objects
    Person list[3] = {
        Person(100, "Homer Simpson", 39),
        Person(110, "Marge Simpson", 36),
        Person(120, "Bart Simpson", 10),
    };

    //Serialize each person in the list - json behaves like a C++ vector
    json simpsons_json;
    for (int i = 0; i < 3; i++) { simpsons_json.push_back(list[i].to_json()); }

    //Print JSON representation of the list
    cout << simpsons_json.dump(4) << endl;

    //Deserialize - Make C++ list from simpsons_json
    Person newList[3];
    for (int i = 0; i < 3; i++) {
        newList[i] = Person::from_json(simpsons_json[i]);
        cout << newList[i].toString() << endl;
    }

    //Save the entire list to a file in JSON format
    ofstream out_file("simpsons_list.json");
    out_file << simpsons_json.dump(4); // No endl needed, as dump(4) includes proper formatting
    out_file.close();

    //Read the list from a file
    ifstream in_file("simpsons_list.json");
    json simpsons_list_json;
    in_file >> simpsons_list_json;
    in_file.close();

    //Show the deserialized list read from the file
    for (int i = 0; i < 3; i++) {
        Person dp = Person::from_json(simpsons_list_json[i]);
        cout << dp.toString() << endl;   //show each person in the list
    }
}
```

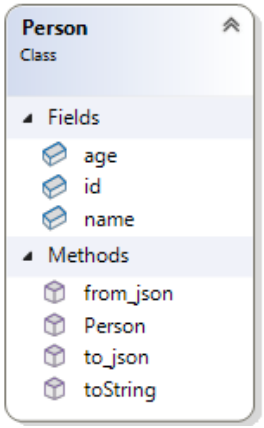


Best Practice
Write/Read lists of JSON objects instead of individual objects.

Example2. Using nlohmann Library

Create a list of objects. Write the list to a file. Read the list from the file.

```
[
    {
        "age": 39,
        "id": 100,
        "name": "Homer Simpson"
    },
    {
        "age": 36,
        "id": 110,
        "name": "Marge Simpson"
    },
    {
        "age": 10,
        "id": 120,
        "name": "Bart Simpson"
    }
]


Person[Id: 100, Name: Homer Simpson, Age: 39]
Person[Id: 110, Name: Marge Simpson, Age: 36]
Person[Id: 120, Name: Bart Simpson, Age: 10]


Person[Id: 100, Name: Homer Simpson, Age: 39]
Person[Id: 110, Name: Marge Simpson, Age: 36]
Person[Id: 120, Name: Bart Simpson, Age: 10]
```



Example2. Using nlohmann Library

Using JSON in C++

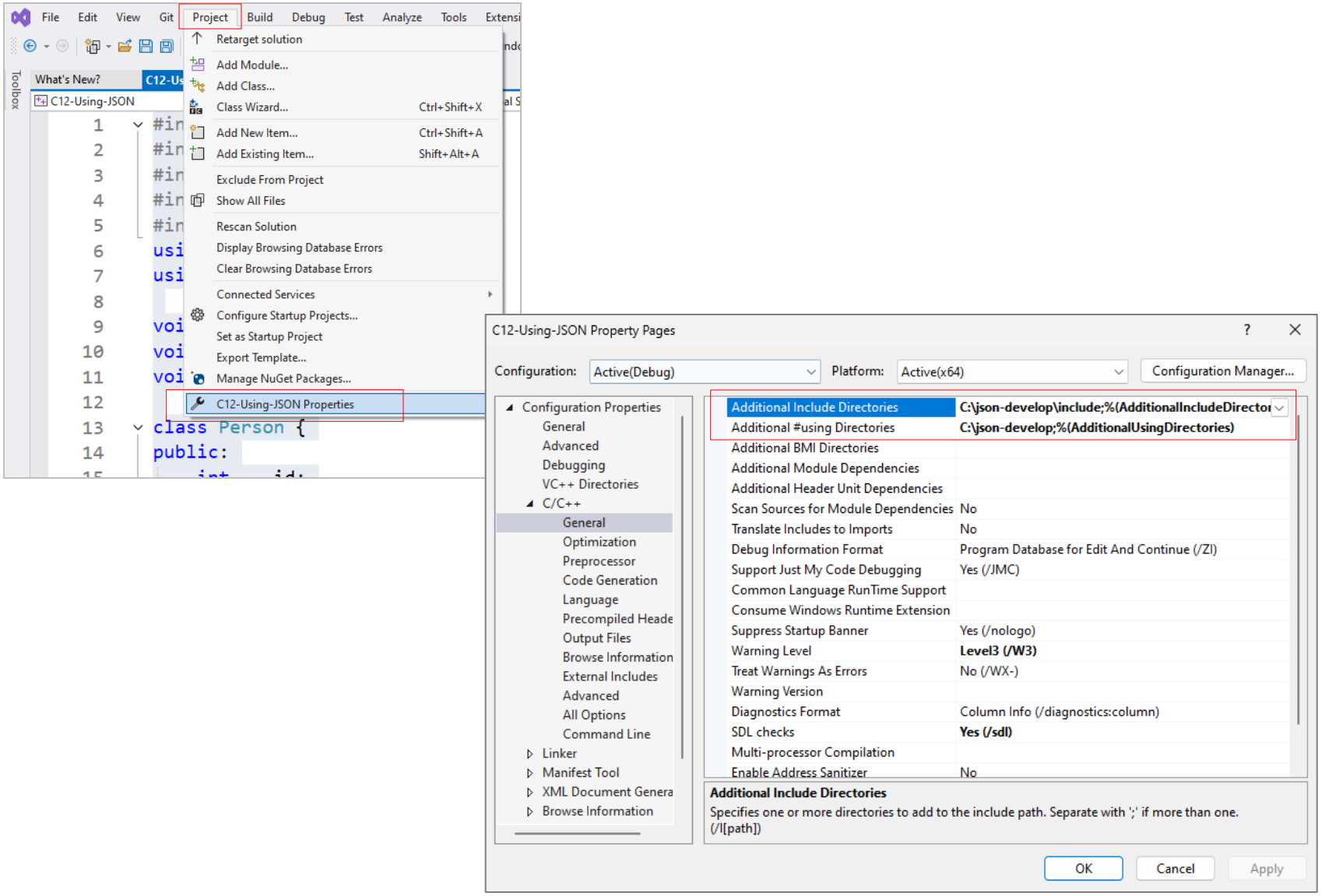
To use JSON in C++, you can use libraries such as nlohmann/json which provides a simple interface for working with JSON.

- Download the `nlohmann` library from https://github.com/nlohmann/json.
- Unzip the file. It should be stored in a folder called `c:/json-develop`.
- Open your project in Visual Studio.
- Add the Include Directory:
  - Right-click on your project in the Solution Explorer and select **Properties**.
  - Go to **Configuration Properties > C/C++ > General**.
  - In the **Additional Include Directories** field, add the path `c:\json-develop\include`.
  - In the **Additional #using Directories** field, add the path `c:\json-develop`

- Include the JSON Header in Your Code:

```
#include <nlohmann/json.hpp>
using json = nlohmann::json;
```

Example2. Using nlohmann Library



Appendix A. Setup **VCPKG** – Package Installer for VS

VCPKG is a C++ package manager developed by Microsoft that simplifies library management in Visual Studio.
Available at www.vcpkg.io

SETUP

Setp1. Install git (Available from https://git-scm.com/downloads/win)

Step2. Install vcpkg – Open a command prompt session. Type the following command

```
git clone https://github.com/microsoft/vcpkg.git
```

Step3. Build vcpkg

```
cd c:\vcpkg
bootstrap-vcpkg.bat
```

Step4. Add c:\vcpkg to the system's path.

Step5. Integrate with Visual Studio

```
vcpkg --integrate install
```

Step6. Install a library. For example,

```
vcpkg install rapidjson
```

Appendix B. Dead References

Set to `nullptr` After Delete (Best Practice – Requires Discipline!)

```
~SomeClass() {
    delete data;
    data = nullptr;  // Mark as invalid
}

int getData() {
    if (data == nullptr) {
        throw runtime_error("Accessing deleted data!");
    }
    return *data;
}
```